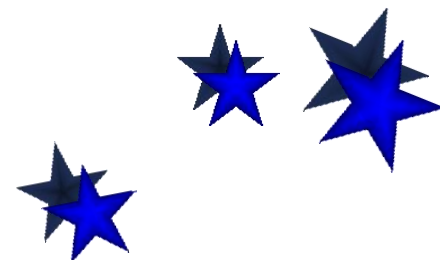


第3章 动态规划

- 动态规划：广泛应用于组合优化问题的算法设计中。
- 组合优化问题：通常有对应的目标函数和约束条件，例如
 - 任务分配问题
 - (1) 输入： $n, m, f_i(x), i=1,2, \dots, n, x=1,2, \dots, m$
 - (2) 输出： n 维向量 $\langle x_1, x_2, \dots, x_n \rangle$, x_i 是第 i 个项目分到的钱数，使得下述条件满足：

约束条件
$$\sum_{i=1}^n x_i = m, \quad x_i \in \mathbb{N}$$

目标函数
$$\max \sum_{i=1}^n f_i(x_i),$$



第3章 动态规划

- 货郎问题（旅行商问题）

- 输入

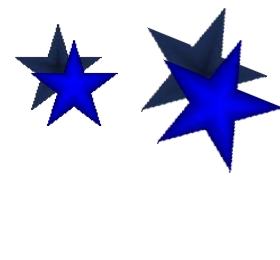
有穷个城市的集合 $C = \{c_1, c_2, \dots, c_n\}$,

距离 $d(c_i, c_j) = d(c_j, c_i) \in \mathbb{Z}^+$, $1 \leq i < j \leq n$

- 输出: $1, 2, \dots, n$ 的一个排列 $k_1, k_2, \dots, k_n$, 使得旅行路线最短。

- 约束条件: $k_1, k_2, \dots, k_n$ 必须构成 $1, 2, \dots, n$ 的排列

- 目标函数: $\min \left\{ \sum_{i=1}^{n-1} d(c_{k_i}, c_{k_{i+1}}) + d(c_{k_n}, c_{k_1}) \right\}$



第3章 动态规划

- 可行解：组合优化问题的解分布在搜索空间中，其中满足约束条件的解成为可行解。
- 最优解：可行解中使目标函数达到最大（或最小）值的解成为最优解。例如：价值最大、代价最小——组合优化问题的最终目标。
- 组合优化问题的搜索空间比较大，中间会有许多重复计算，通常是指数量级的（ $n!$ 、 2^n ）。
 - 蛮力算法：大规模问题是无法求解的。
 - 动态规划：
 - (1) 将问题的求解过程划分成一个多阶段决策的过程。每阶段都对应原问题的一个子问题。



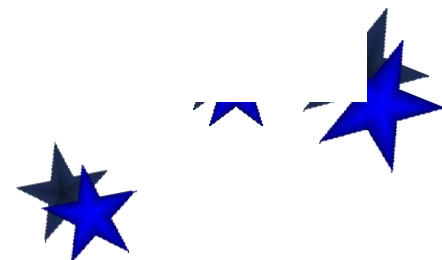
第3章 动态规划

(2) 算法划分出子问题的边界，从小的子问题开始，逐层向上求解。

(3) 通过子问题间的依赖关系，有效利用前面已经产生的结果，最大限度减少重复工作，提高算法效率。

(4) 很多蛮力算法是指数时间的问题，采用动态规划技术将时间复杂度降低到 n^2 、 n^3 等多项式级别。

(5) 不足：需要较大的存储空间存储子问题计算的中间结果，对于输入规模很大的情况，这个代价可能比时间的消耗更难以承受。



第3章 动态规划

3.1 动态规划的设计思想

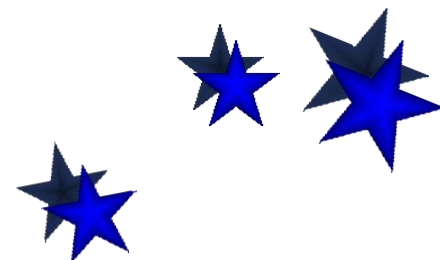
- 实例——斐波那契数列、最短路径问题
- 使用动态规划技术的必要条件

3.2 动态规划算法的设计要素

- 子问题的划分和递推方程——矩阵链相乘
- 动态规划算法的实现——递归、迭代

2.3 动态规划算法的典型应用

- 投资问题
- 背包问题

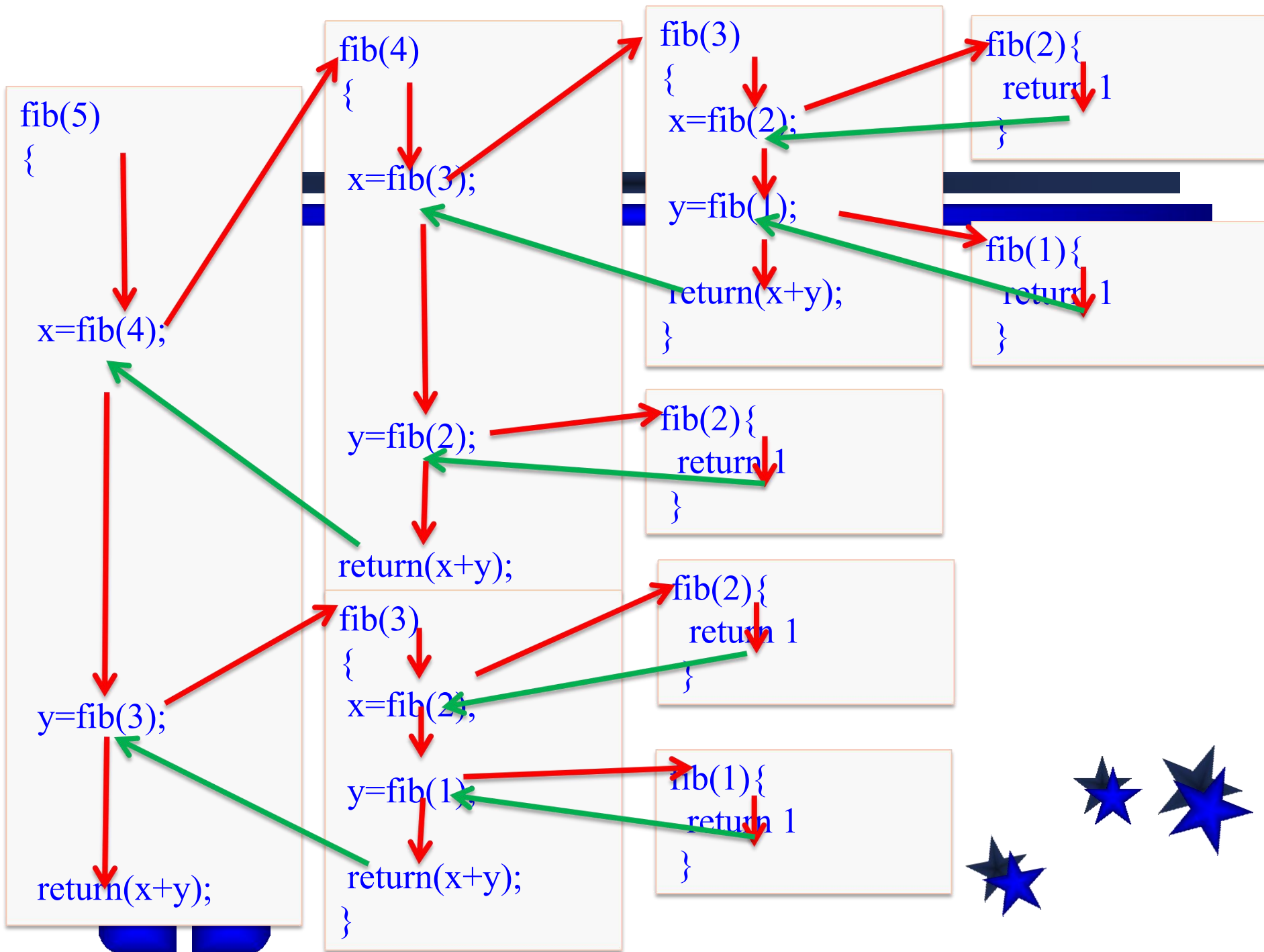


3.1 动态规划的设计思想

■ 从求解斐波那契数列看动态规划法

```
int count=1;           //累计调用的步骤
int fib(int n)          //算法
{ printf("(%d)求解Fib(%d)\n", count++, n);
  if (n==1 || n==2)
  { printf(" Fib(%d)=%d\n",n,1);
    return 1;
  }
  else
  { int x=fib(n-1);
    int y=fib(n-2);
    printf(" Fib(%d)=Fib(%d)+Fib(%d)=%d\n",n,n-1,n-2,x+y);
    return x+y;
  }
}
```





Fib(5)时的输出结果:

(1) 求解Fib(5)

(2) 求解Fib(4)

(3) 求解Fib(3)

(4) 求解Fib(2)

$\text{Fib}(2)=1$

(5) 求解Fib(1)

$\text{Fib}(1)=1$

$\text{Fib}(3)=\text{Fib}(2)+\text{Fib}(1)=2$

(6) 求解Fib(2)

$\text{Fib}(2)=1$

$\text{Fib}(4)=\text{Fib}(3)+\text{Fib}(2)=3$

(7) 求解Fib(3)

(8) 求解Fib(2)

$\text{Fib}(2)=1$

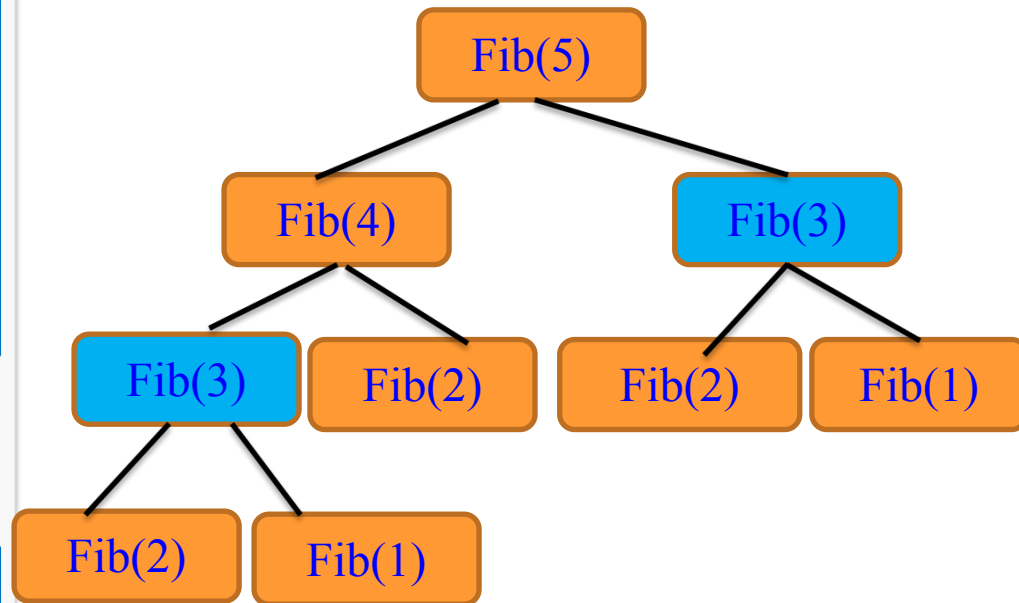
(9) 求解Fib(1)

$\text{Fib}(1)=1$

$\text{Fib}(3)=\text{Fib}(2)+\text{Fib}(1)=2$

$\text{Fib}(5)=\text{Fib}(4)+\text{Fib}(3)=5$

自顶向下的求解过程:



计算过程中存在着重复计算，例如：存在两次重复计算Fib(3)值的情况。

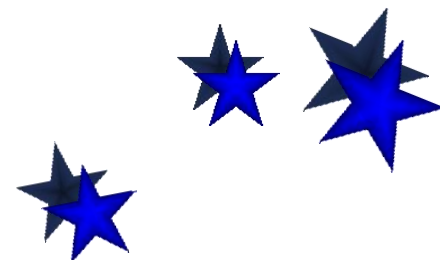


为此**避免重复设计**，设计一个 dp 数组， $dp[i]$ 存放Fib(i)的值，首先设置 $dp[1]$ 和 $dp[2]$ 均为1，再让 i 从3到 n 循环以计算 $dp[3]$ 到 $dp[n]$ 的值，最后返回 $dp[n]$ 即 $fib(n)$ 。对应的算法如下：

```
int dp[MAX];
int count=1;
int fib2(int n)
{ dp[1]=dp[2]=1;
  printf("(%d)Fib(1)=1\n",count++);
  printf("(%d)Fib(2)=1\n",count++);
  for (int i=3;i<=n;i++)
  { dp[i]=dp[i-1]+dp[i-2];
    printf("(%d)Fib(%d)=%d\n",count++,i,dp[i]);
  }
  return dp[n];
}
```

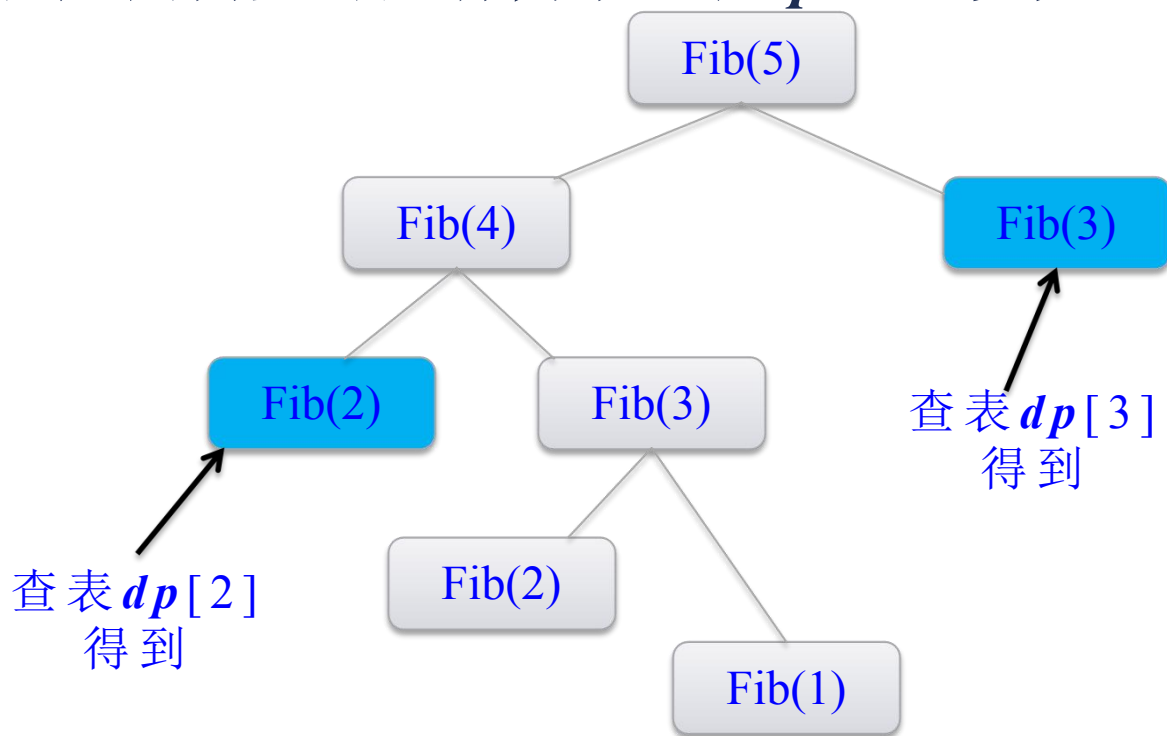
执行fib(5)时的输出如下

```
(1) fib(1)=1
(2) fib(2)=1
(3) fib(3)=2
(4) fib(4)=3
(5) fib(5)=5
```

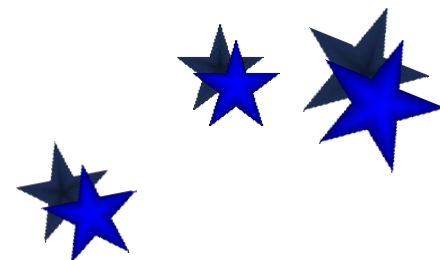


3.1 动态规划的设计思想

- 其执行过程为**自底向上**，即先求出子问题解，将计算结果存放在一张表中(数组 dp)，且相同的子问题只计算一次，在后面需要时只有简单查表 dp ，避免大量的重复计算。

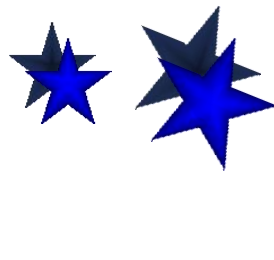
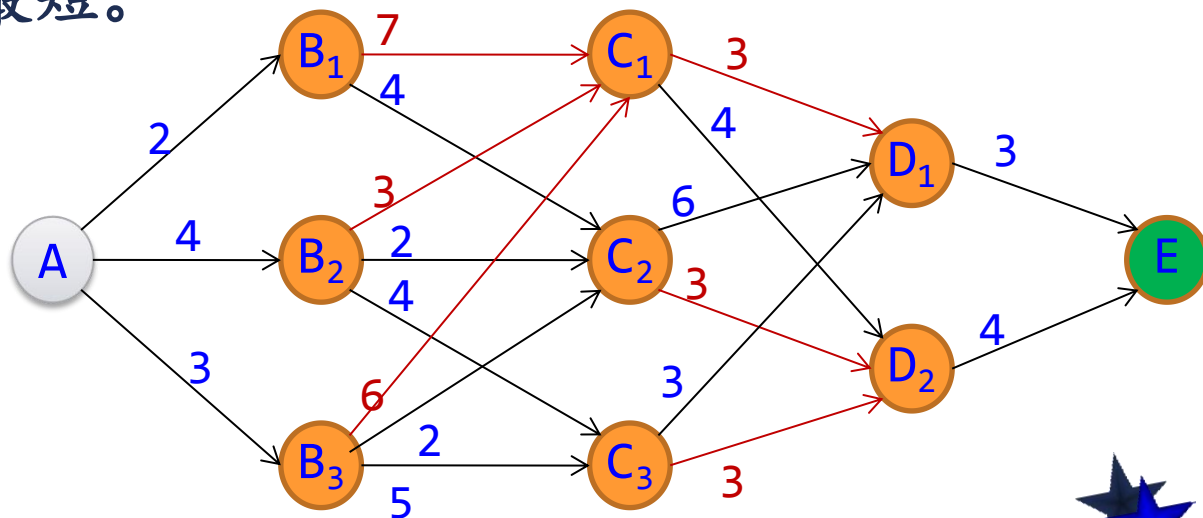


备忘录方法

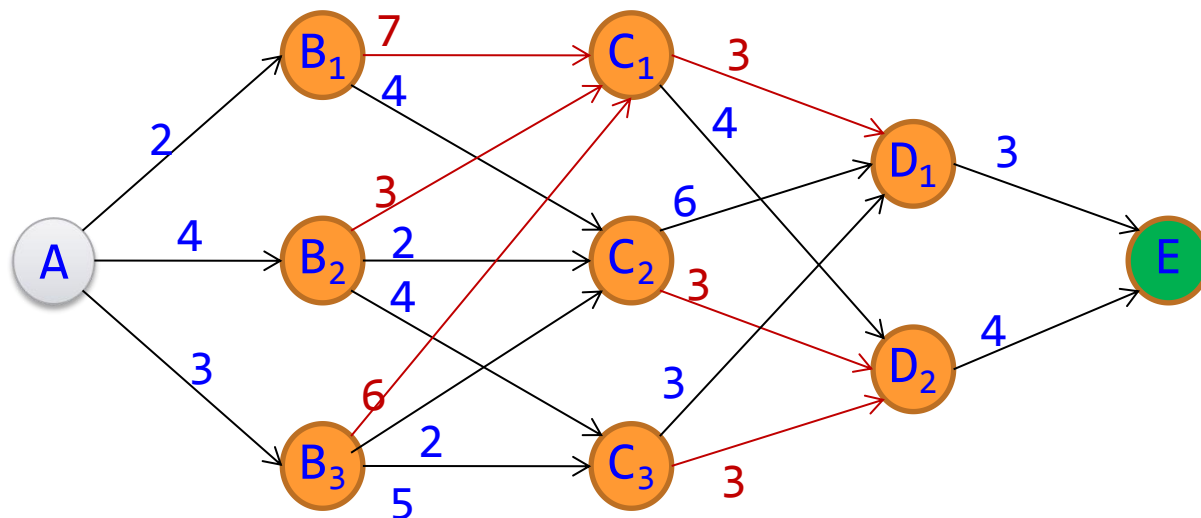


3.1 动态规划的设计思想

- 动态规划是解决多阶段决策问题的优化方法。把多阶段转化为一系列单阶段问题，利用各阶段之间的关系，逐个求解。
- 举例：在A处有一水库，现需要从A点铺设一条管道到E点。边上的数字表示与其相连的两个地点之间所需修建的管道长度。现要找出一条从A到E的修建线路，使得所需修建的管道长度最短。

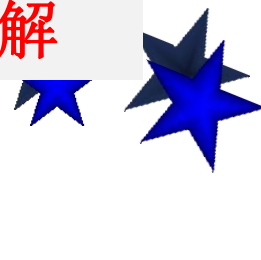


3.1 动态规划的设计思想

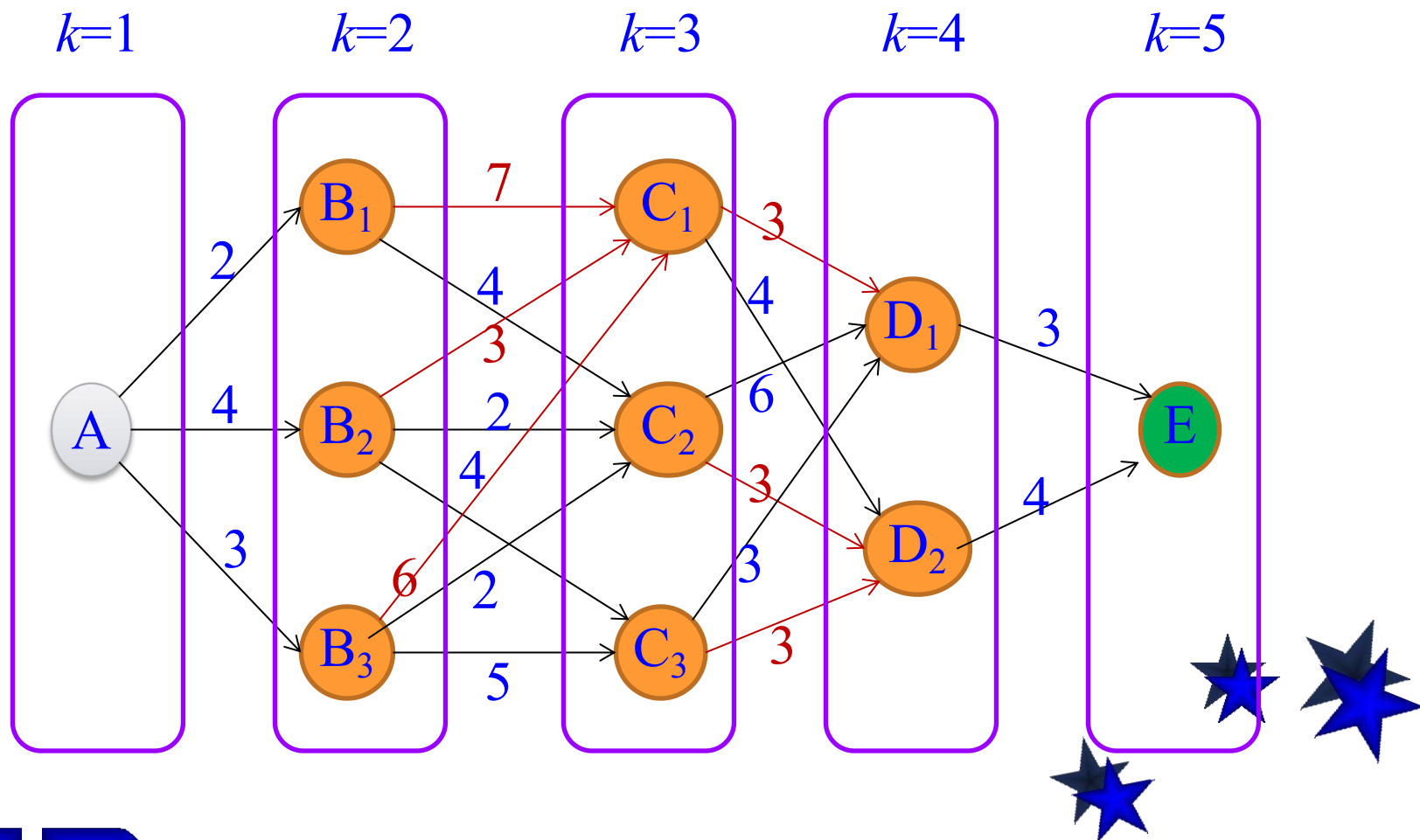


- 使用贪婪算法求解该问题，得到的解为：
 - (1) 顺序: $A \rightarrow B_1 \rightarrow C_2 \rightarrow D_2 \rightarrow E$ (13)
 - (2) 逆序: $A \rightarrow B_2 \rightarrow C_1 \rightarrow D_1 \rightarrow E$ (13)
- 真正的最优解是: $A \rightarrow B_3 \rightarrow C_2 \rightarrow D_2 \rightarrow E$ (12)

贪婪算法无法
得到最优解



在从 $A \sim E$ 的过程中，依据按位置所做决策的次数及所做决策的先后次序，**将问题分为5个阶段**，变量 k 用于表示各阶段取值为1~5，图中第5阶段是虚拟的一个边界阶段。每个阶段中包含若干状态结点。



3.1 动态规划的设计思想

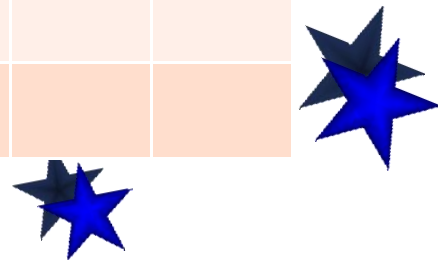
■ 动态规划问题的解法：逆序解法、正序解法

➤ **逆序解法**：从终点向起点逐个阶段计算。

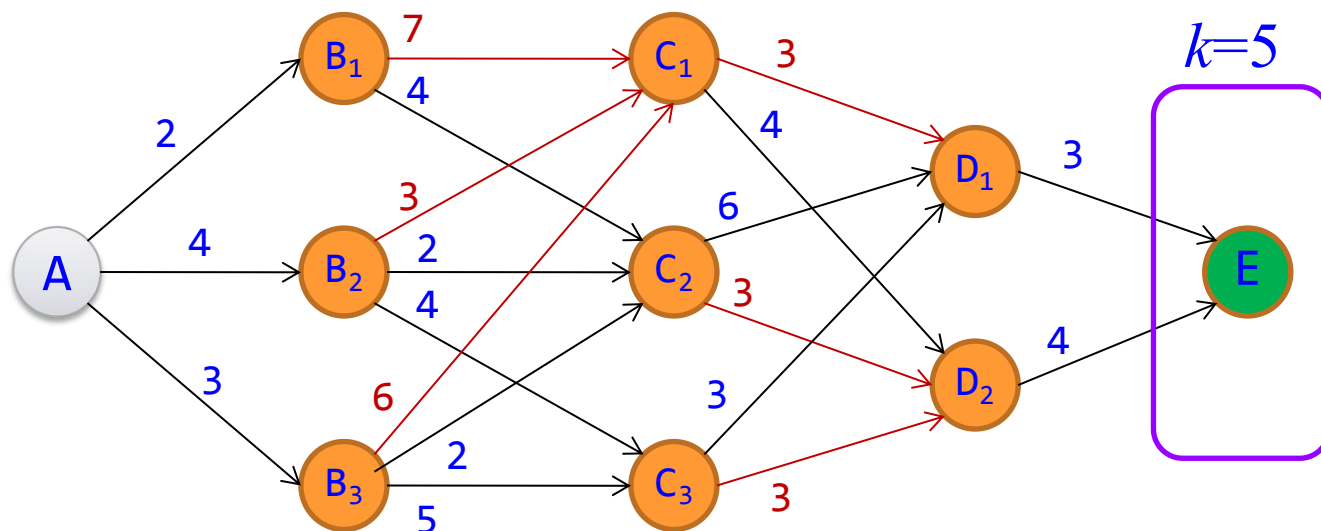
(1) 数组 dp ：用于记录每个阶段中各状态结点到终点的最短距离。

(2) 数组 $next$ ：记录各状态结点到终点的最短距离的后继结点编号

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
dp										
$Next$										



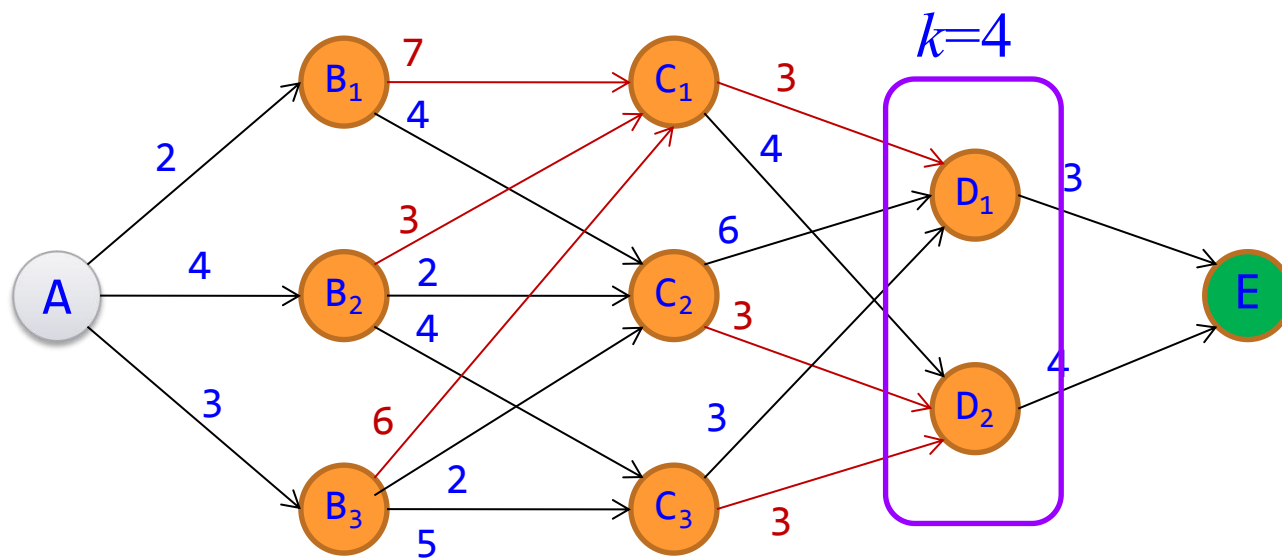
3.1 动态规划的设计思想



① 第5阶段作为起始阶段：状态***E***到终点的最短距离为0

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>										0
<i>Next</i>										



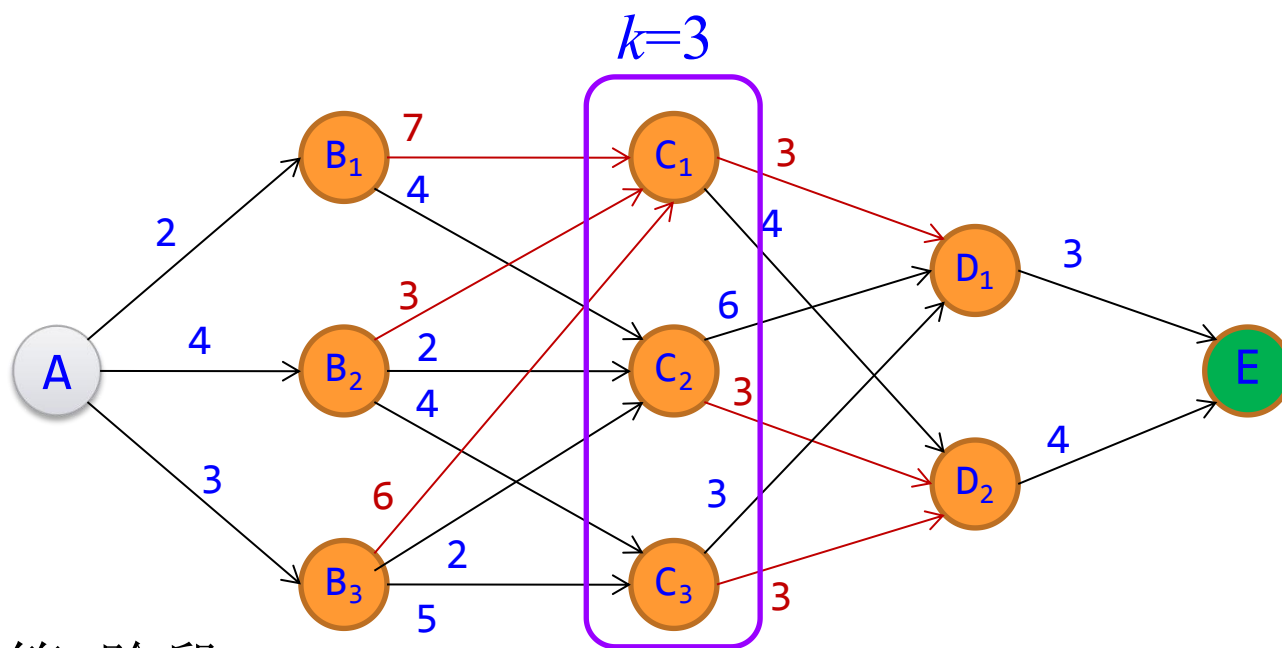


② 第4阶段:

- $D_1 = \min \{ \text{Dist}(D_1, E) + dp[E] \} = \{ 3 + 0 \} = 3$
- $D_2 = \min \{ \text{Disu}(D_2, E) + dp[E] \} = \{ 4 + 0 \} = 4$

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>								3	4	0
<i>Next</i>								9	9	





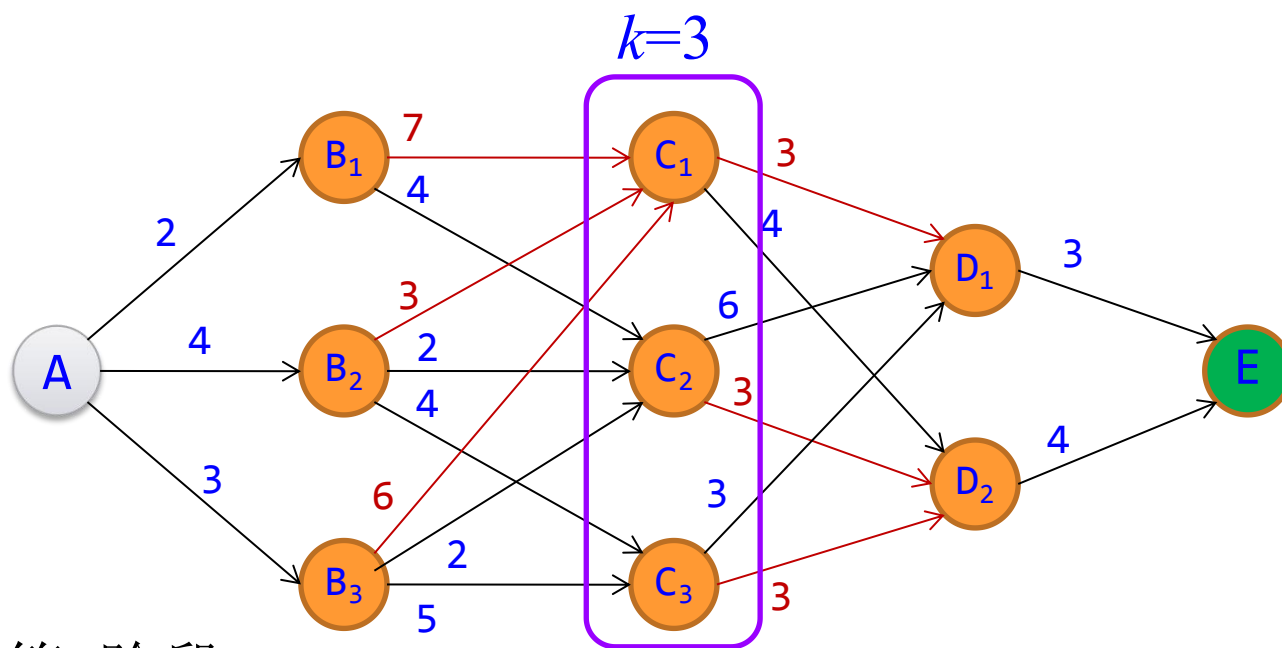
③ 第3阶段:

- $$C_1 = \min \{ \text{Dist}(C_1, D_1) + dp[D_1], \text{Dist}(C_1, D_2) + dp[D_2] \}$$

$$= \min \{ 3+3, 4+4 \} = 6$$

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>					6			3	4	0
<i>Next</i>					7			9	9	





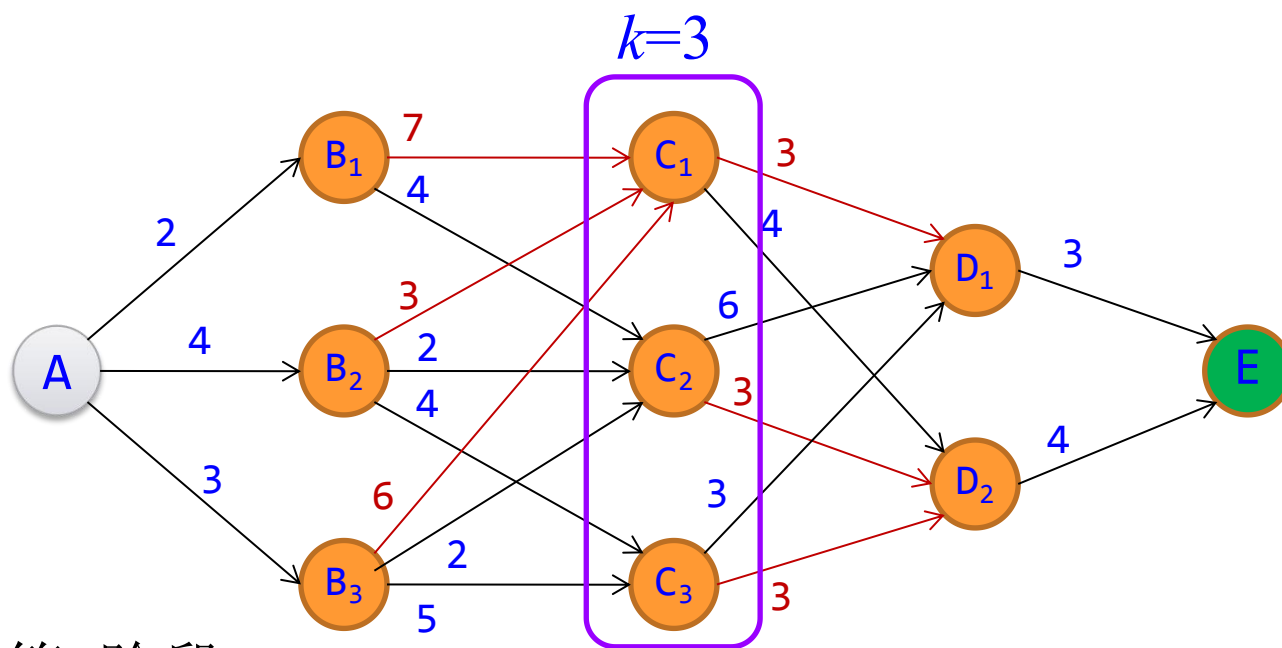
③ 第3阶段:

- $$C_2 = \min \{ \text{Dist}(C_2, D_1) + dp[D_1], \text{Dist}(C_2, D_2) + dp[D_2] \}$$

$$= \min \{ 6 + 3, 3 + 4 \} = 7$$

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
dp					6	7		3	4	0
$Next$					7	8		9	9	





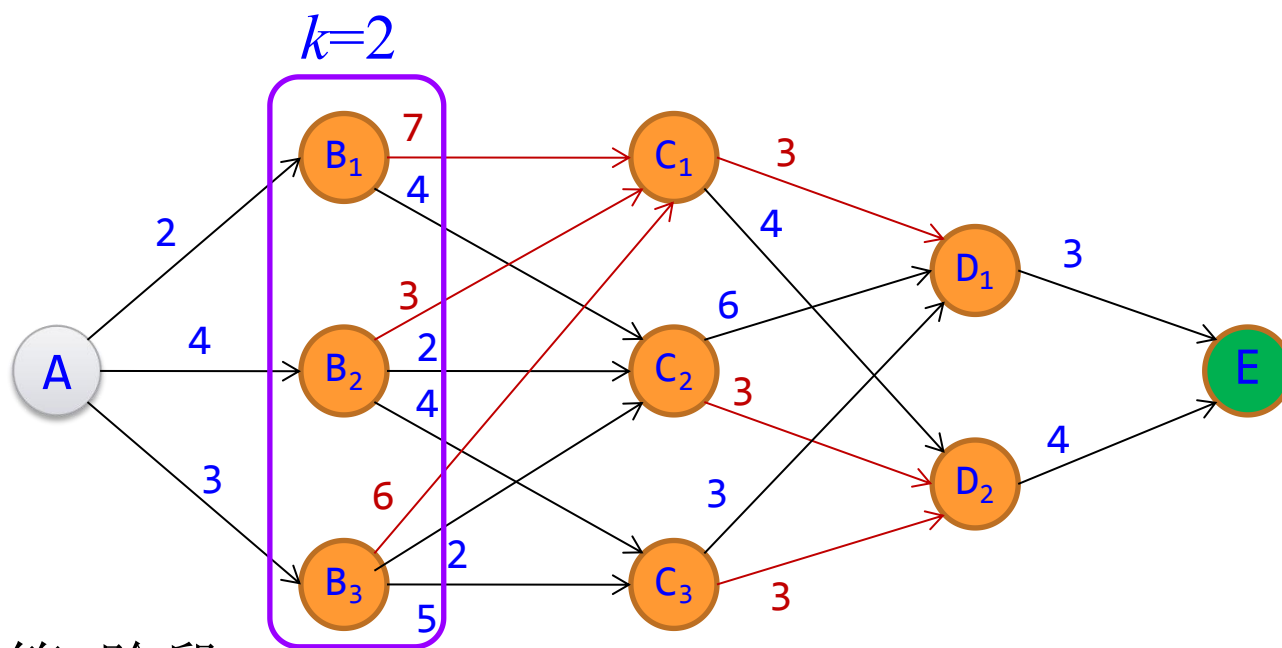
③ 第3阶段:

- $$C_3 = \min \{ \text{Dist}(C_3, D_1) + dp[D_1], \text{Dist}(C_3, D_2) + dp[D_2] \}$$

$$= \min \{ 3+3, 3+4 \} = 6$$

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>					6	7	6	3	4	0
<i>Next</i>					7	8	7	9	9	





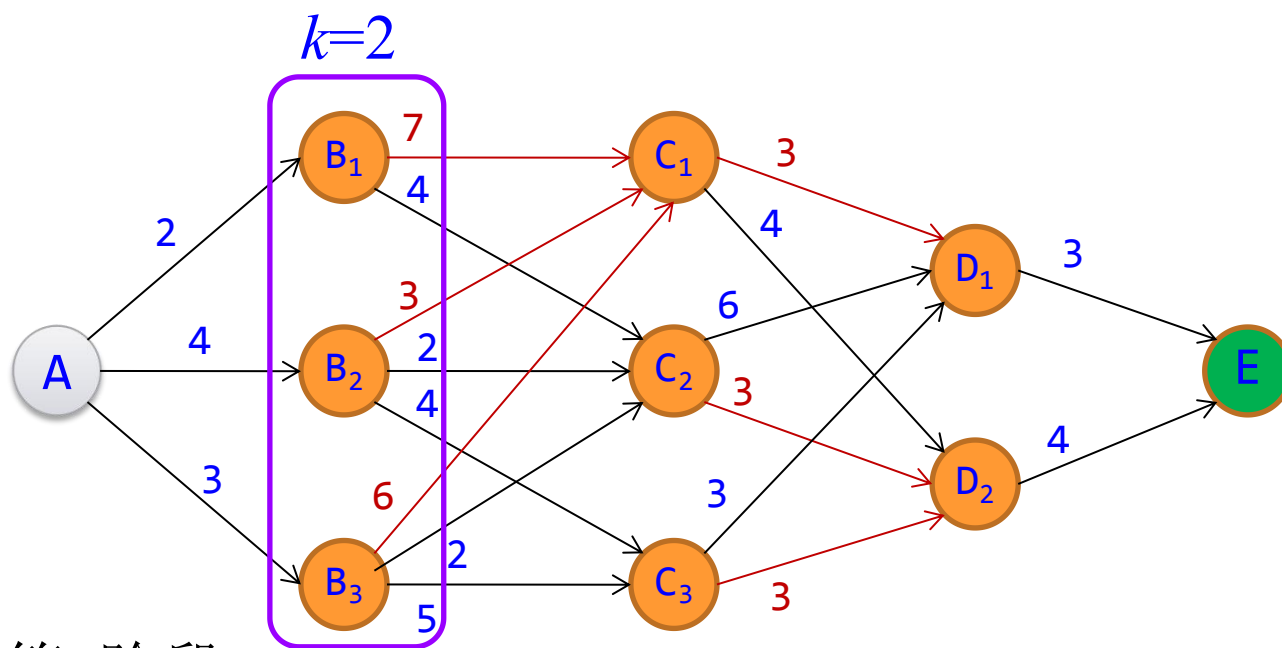
④ 第2阶段:

- $$B_1 = \min \{ \text{Dist}(B_1, C_1) + dp[C_1], \text{Dist}(B_1, C_2) + dp[C_2] \}$$

$$= \min \{ 7 + 6, 4 + 7 \} = 11$$

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>		11			6	7	6	3	4	0
<i>Next</i>		5			7	8	7	9	9	



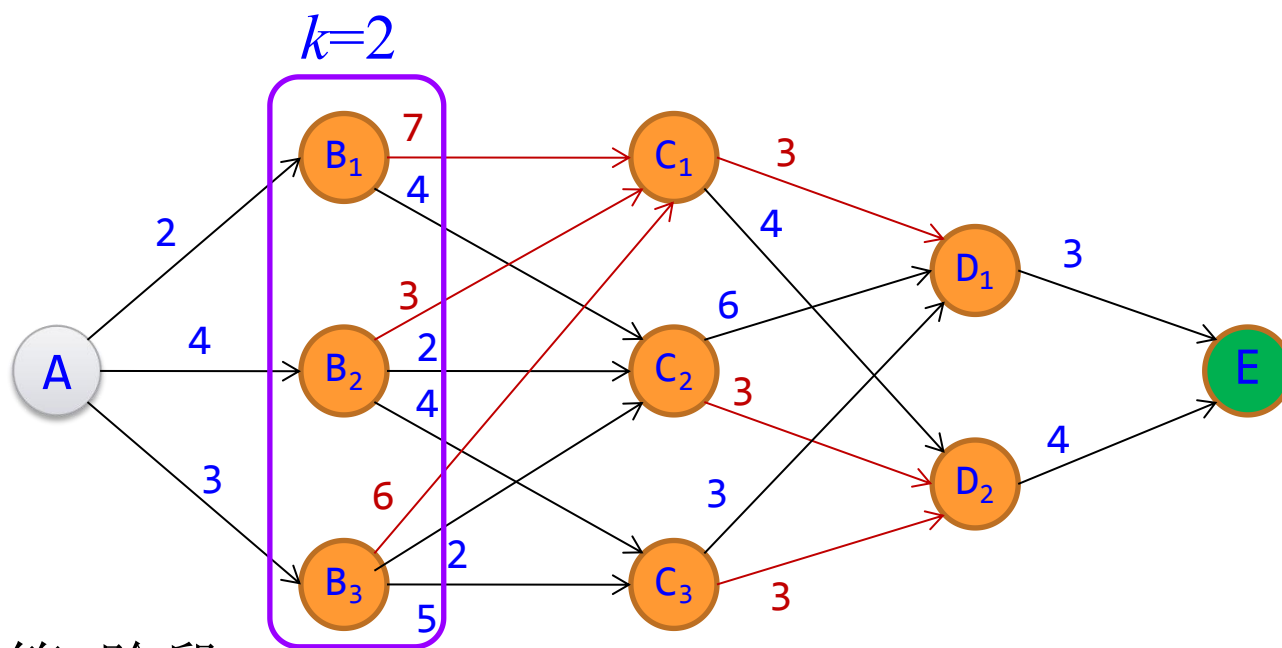


④ 第2阶段:

- $$B_2 = \min \{ \text{Dist}(B_2, C_1) + dp[C_1], \text{Dist}(B_2, C_2) + dp[C_2], \text{Dist}(B_2, C_3) + dp[C_3] \} = \min \{ 3+6, 2+7, 4+6 \} = 9$$

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>		11	9		6	7	6	3	4	0
<i>Next</i>		5	4		7	8	7	9	9	



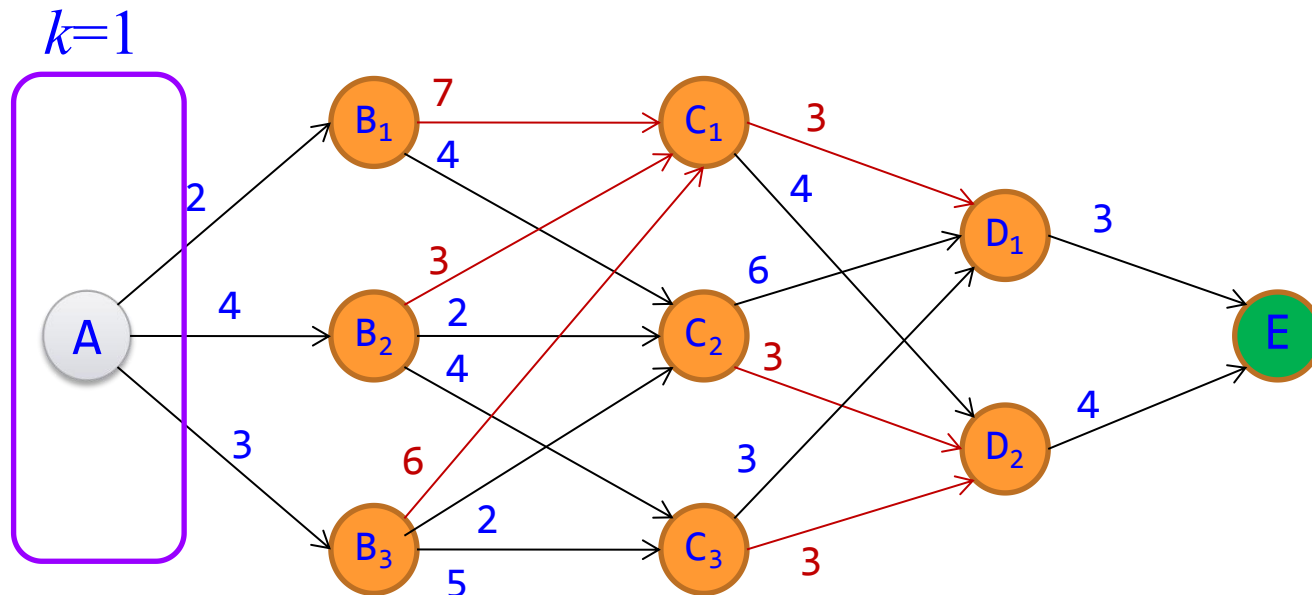


④ 第2阶段:

- $$B_3 = \min \{ \text{Dist}(B_3, C_1) + dp[C_1], \text{Dist}(B_3, C_2) + dp[C_2], \text{Dist}(B_3, C_3) + dp[C_3] \} = \min \{ 6+6, 2+7, 5+6 \} = 9$$

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>		11	9	9	6	7	6	3	4	0
<i>Next</i>		5	4	5	7	8	7	9	9	





⑤ 第1阶段:

- $$A = \min \{ \text{Dist}(A, B_1) + dp[B_1], \\ \text{Dist}(A, B_2) + dp[B_2], \\ \text{Dist}(A, B_3) + dp[B_3] \} = \min \{ 2+11, 4+9, 3+9 \} = 12$$

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>	12	11	9	9	6	7	6	3	4	0
<i>Next</i>	3	5	4	5	7	8	7	9	9	



3.1 动态规划的设计思想

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>	12	11	9	9	6	7	6	3	4	0
<i>Next</i>	3	5	4	5	7	8	7	9	9	

● 结果输出

(1) $dp[0]$ 中就是问题的最优解，即A→E的最短路径为12。

(2) 路径为： A→Next[0](3,B3)→Next[3](5,C2)→
Next[5](8,D2)→Next[8](9,E)。

A→B3→C2→D2→E



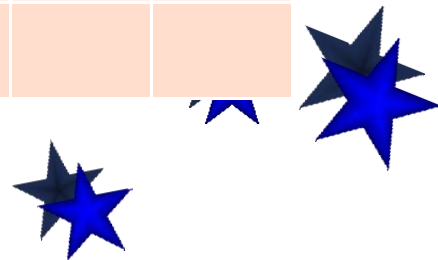
3.1 动态规划的设计思想

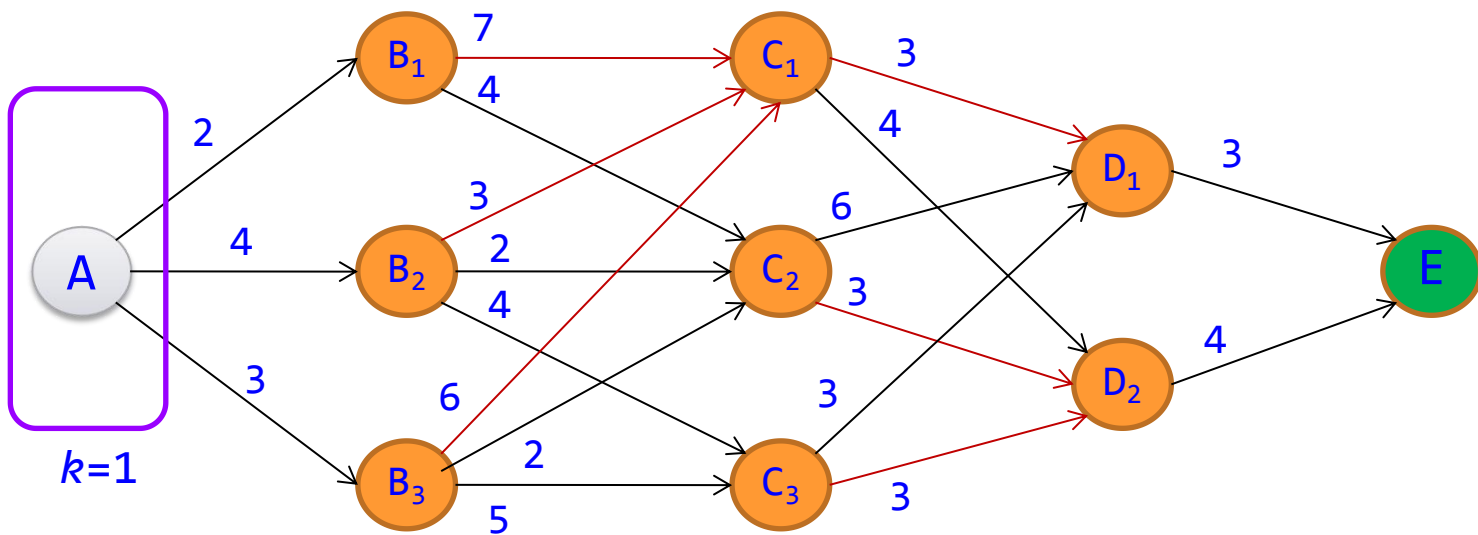
➤ **正序解法：** 从起点向终点逐个阶段计算。

(1) 数组 dp ： 用于记录起点到每个阶段中各状态结点的最短距离。

(2) 数组 pre ： 记录各状态结点到起点的最短距离的前趋结点编号

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
dp										
pre										

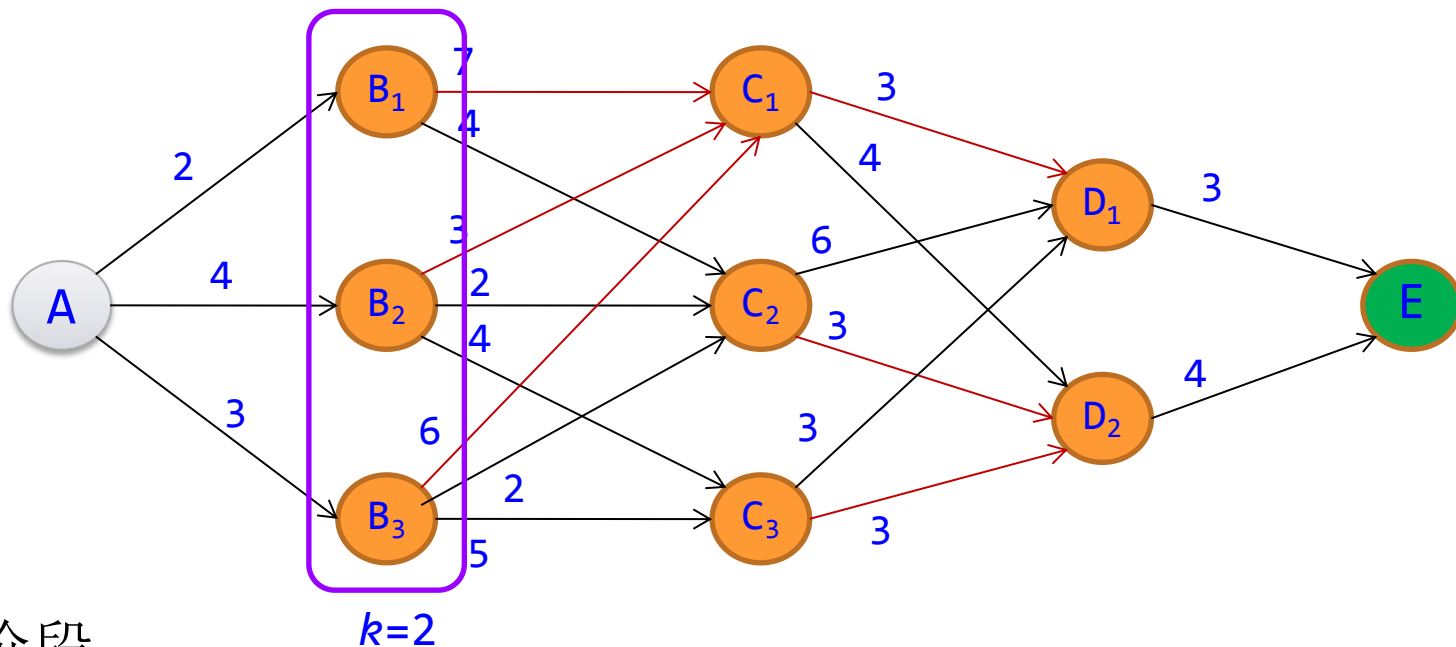




① 第1阶段作为起始阶段：状态A到起点的最短距离为0

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>	0									
<i>pre</i>										



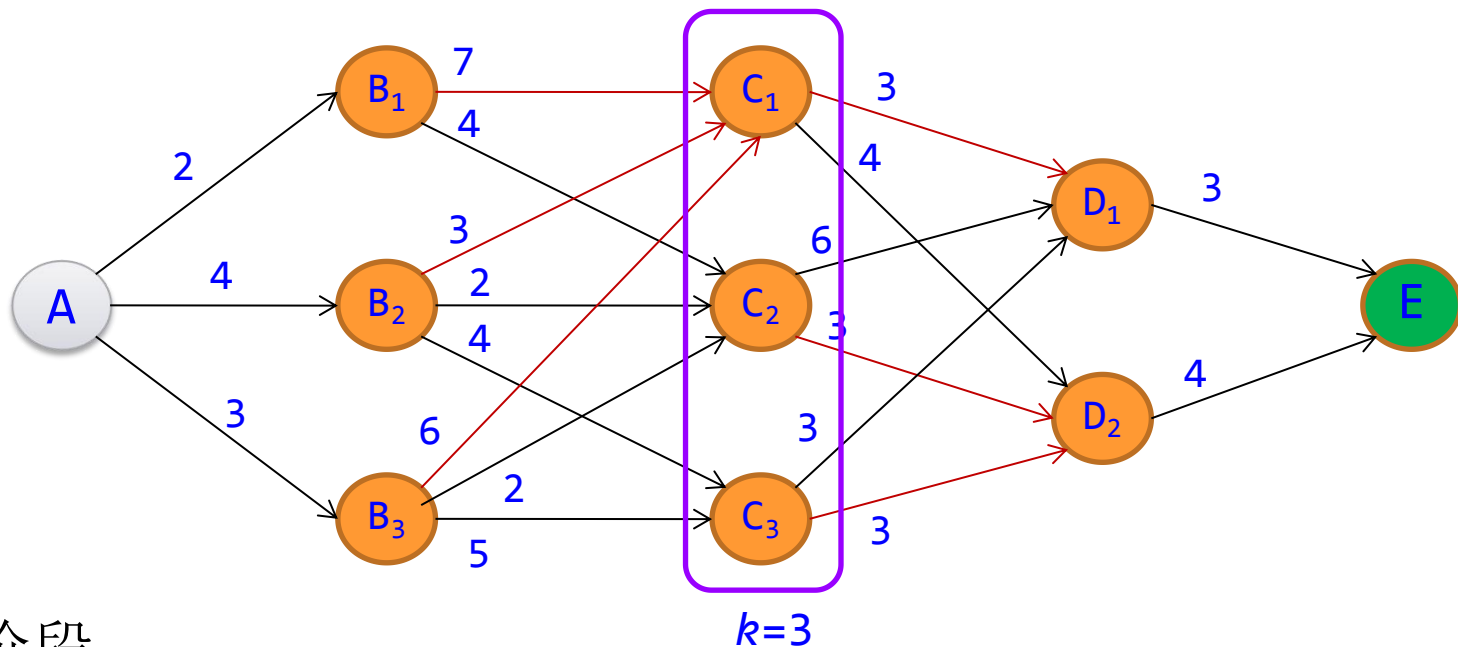


② 第2阶段:

- $B1 = \min \{ dp[A] + \text{Dist}(A, B_1) \} = \{ 0 + 2 \} = 2$
- $B2 = \min \{ dp[A] + \text{Dist}(A, B_2) \} = \{ 0 + 4 \} = 4$
- $B3 = \min \{ dp[A] + \text{Dist}(A, B_3) \} = \{ 0 + 3 \} = 3$

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>	0	2	4	3						
<i>pre</i>		0	0	0						



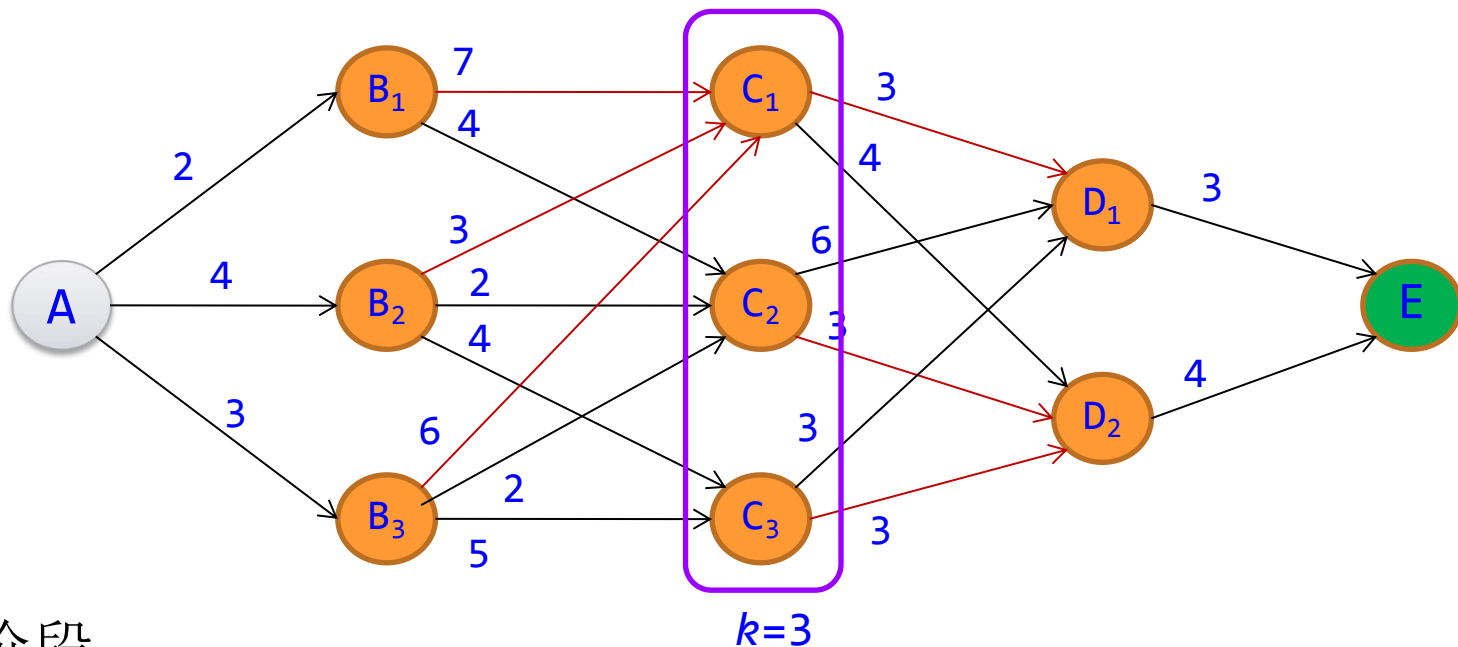


③ 第3阶段:

- $C1 = \min \{ dp[B_1] + \text{Dist}(B_1, C_1),$
 $dp[B_2] + \text{Dist}(B_2, C_1),$
 $dp[B_3] + \text{Dist}(B_3, C_1), \} = \{2+7, 4+3, 3+6\} = 7$

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>	0	2	4	3	7					
<i>pre</i>		0	0	0	2					



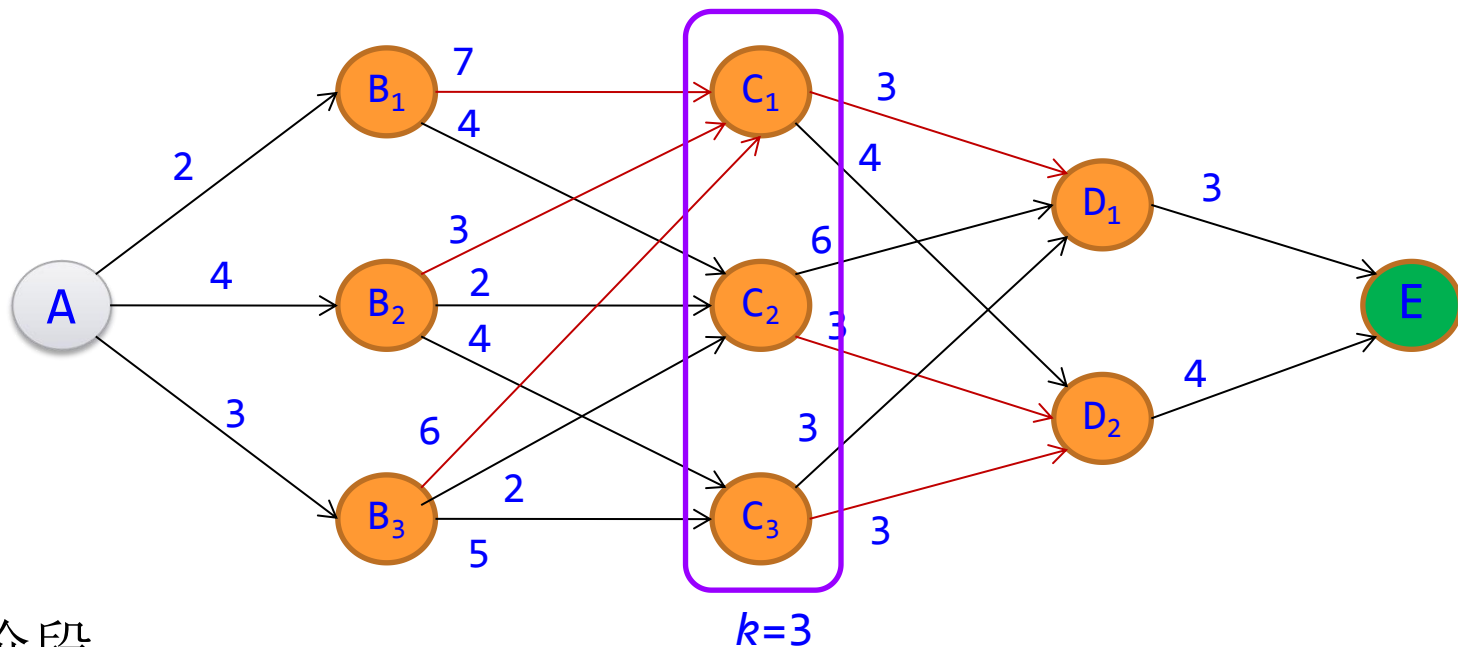


③ 第3阶段:

- $C_2 = \min \{ dp[B_1] + \text{Dist}(B_1, C_2), dp[B_2] + \text{Dist}(B_2, C_2), dp[B_3] + \text{Dist}(B_3, C_2), \} = \{ 2+4, 4+2, 3+2 \} = 5$

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>	0	2	4	3	7	5				
<i>pre</i>		0	0	0	2	3				

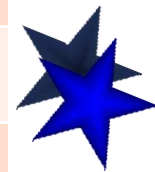


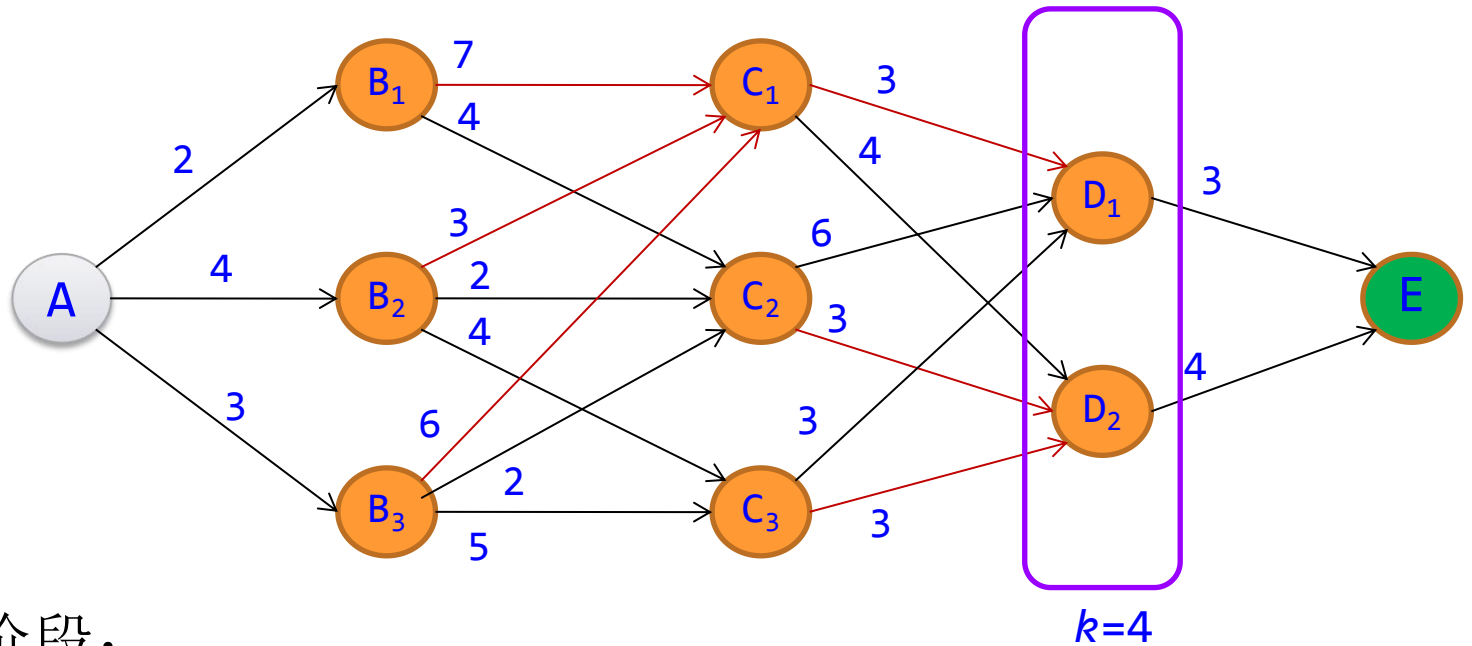


③ 第3阶段:

- $C_3 = \min \{ dp[B_2] + \text{Dist}(B_2, C_3), dp[B_3] + \text{Dist}(B_3, C_3), \} = \{ 4 + 4, 3 + 5 \} = 8$

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>	0	2	4	3	7	5	8			
<i>pre</i>		0	0	0	2	3	2			



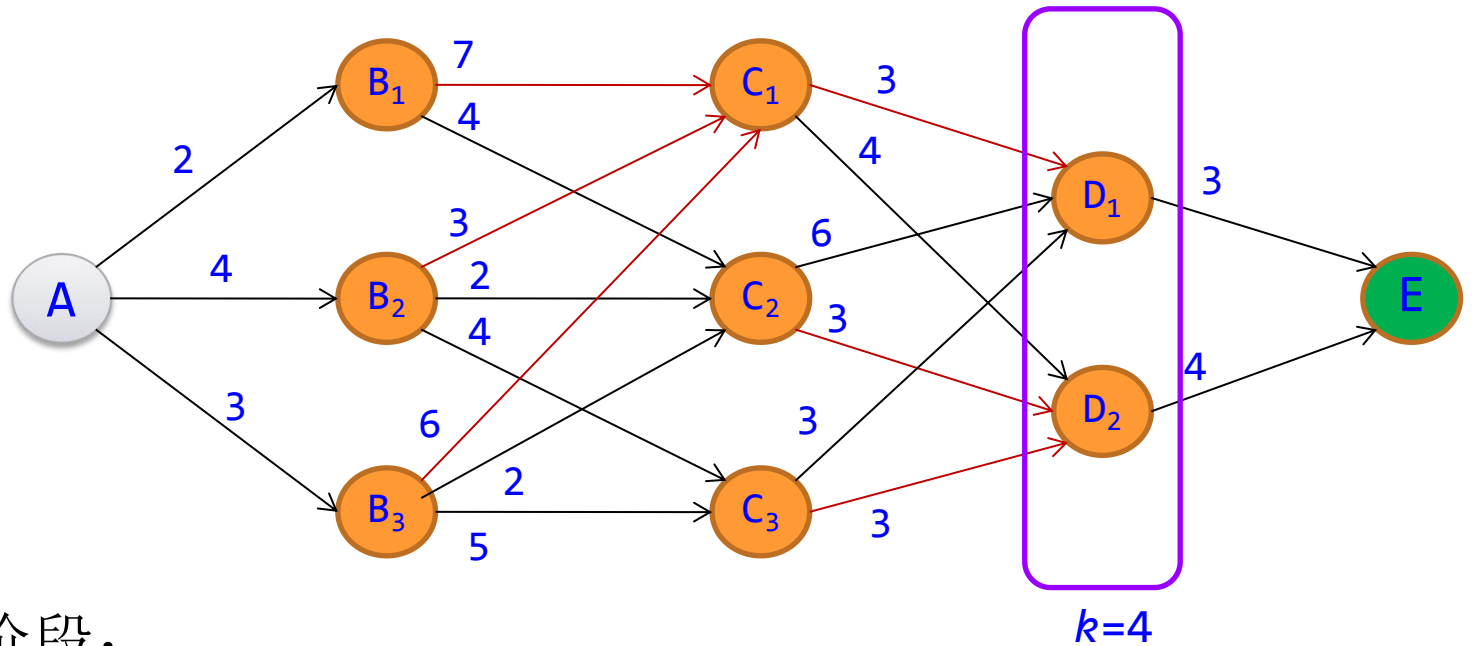


④ 第4阶段:

- $D_1 = \min \{ dp[C_1] + \text{Dist}(C_1, D_1),$
 $dp[C_2] + \text{Dist}(C_2, D_1),$
 $dp[C_3] + \text{Dist}(C_3, D_1) \} = \{ 7+3, 5+6, 8+3 \} = 10$

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>	0	2	4	3	7	5	8	10		
<i>pre</i>		0	0	0	2	3	2	4		



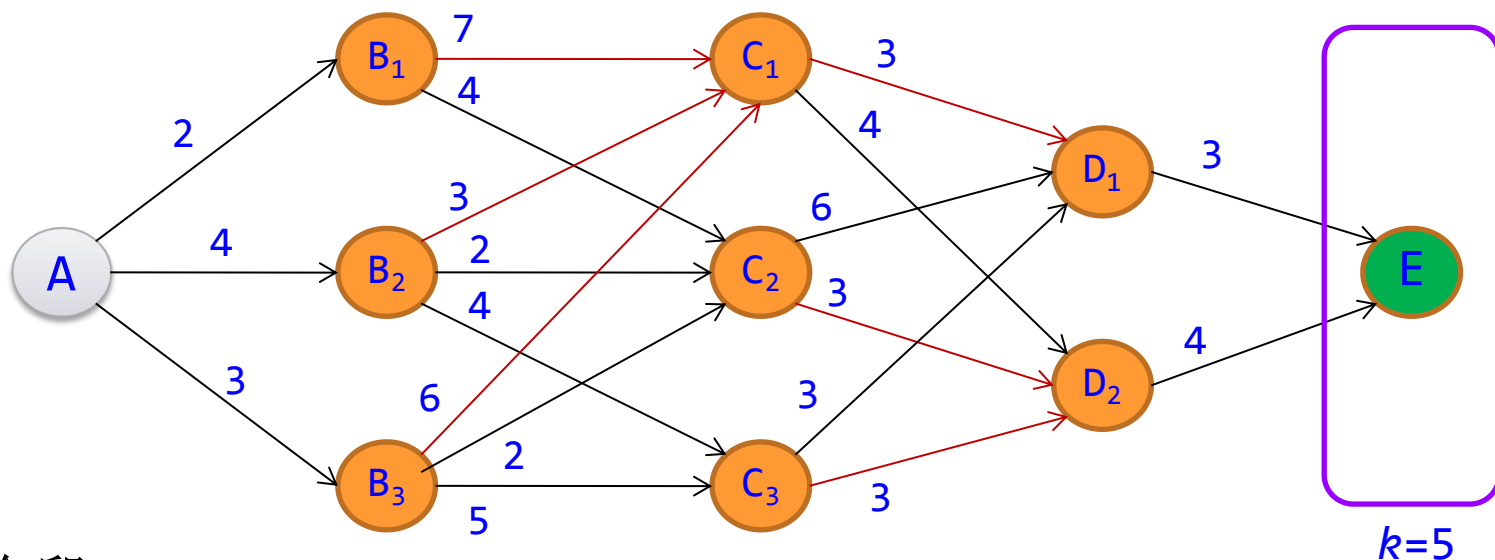


④ 第4阶段:

- $$D_2 = \min \{ dp[C_1] + \text{Dist}(C_1, D_2), \\ dp[C_2] + \text{Dist}(C_2, D_2), \\ dp[C_3] + \text{Dist}(C_3, D_2) \} = \{ 7+4, 5+3, 8+3 \} = 8$$

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>	0	2	4	3	7	5	8	10	8	
<i>pre</i>		0	0	0	2	3	2	4	5	





⑤ 第5阶段:

- $$E = \min \{ dp[D_1] + \text{Dist}(C_1, D_2), dp[D_2] + \text{Dist}(C_2, D_2) \}$$

$$= \{10+3, 8+4\} = 12$$

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>	0	2	4	3	7	5	8	10	8	12
<i>pre</i>		0	0	0	2	3	2	4	5	8



3.1 动态规划的设计思想

	A	B1	B2	B3	C1	C2	C3	D1	D2	E
	0	1	2	3	4	5	6	7	8	9
<i>dp</i>	0	2	4	3	7	5	8	10	8	12
<i>pre</i>		0	0	0	2	3	2	4	5	8

● 结果输出

(1) $dp[9]$ 中就是问题的最优解即A→E的最短路径为12。

(2) 路径为: $E \leftarrow pre[9](8, D2) \leftarrow pre[8](5, C2) \leftarrow pre[5](3, B3) \leftarrow Next[3](0, A)$ 。

$E \leftarrow D2 \leftarrow C2 \leftarrow B3 \leftarrow A$



3.1 动态规划的设计思想

➤ 例题：多起点、多终点的最短路径问题——书中例子
问题：

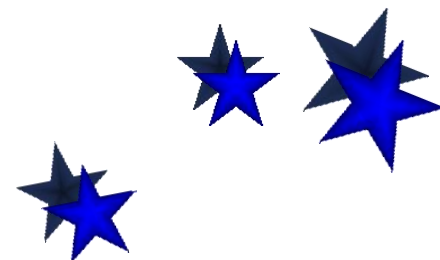
输入：起点集合 $S\{S_1, S_2, \dots, S_n\}$,

终点集合 $T\{T_1, T_2, \dots, T_m\}$,

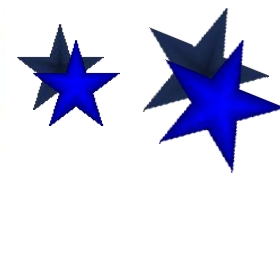
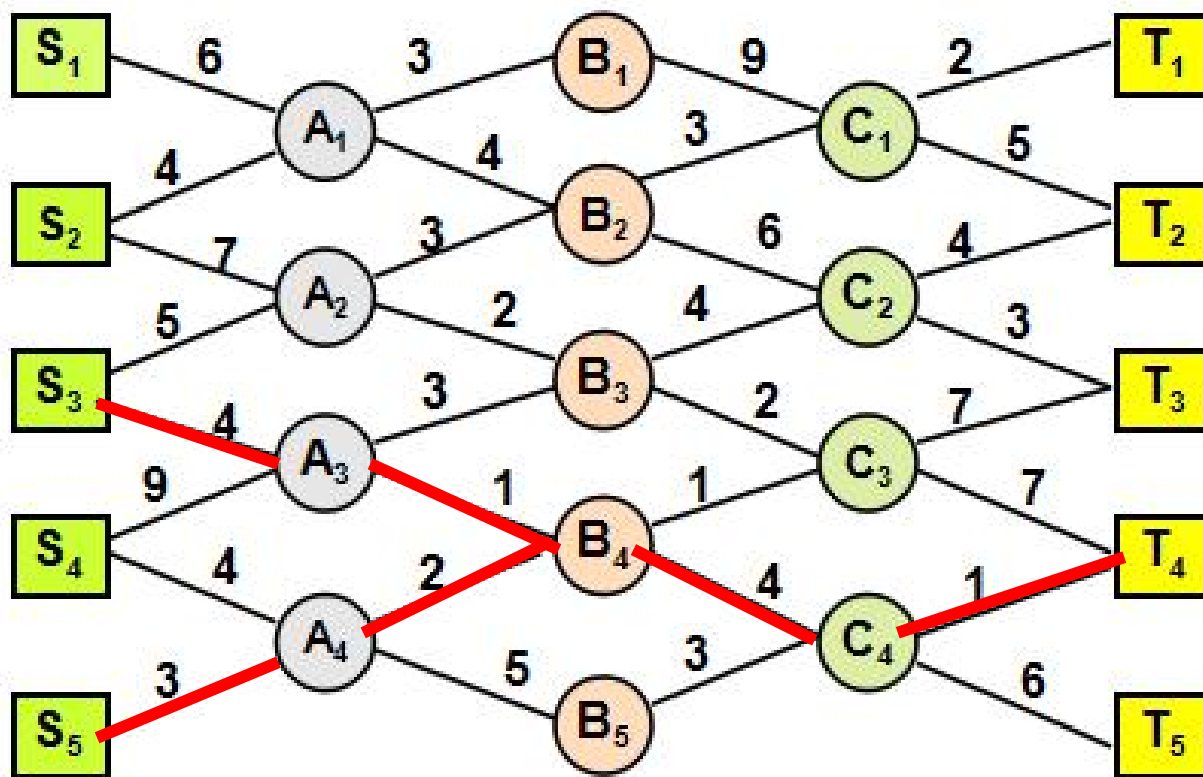
中间结点集 A 、 B 、 C ,

边集 E , 对于任意边 e 有长度

输出：一条从起点到终点的最短路径



3.1 动态规划的设计思想



3.1 动态规划的设计思想

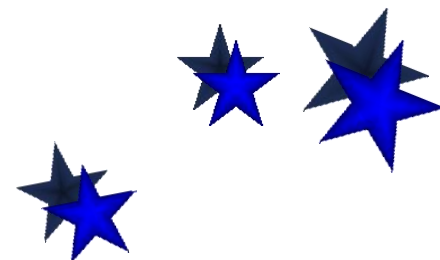
算法设计：

(1) 蛮力算法

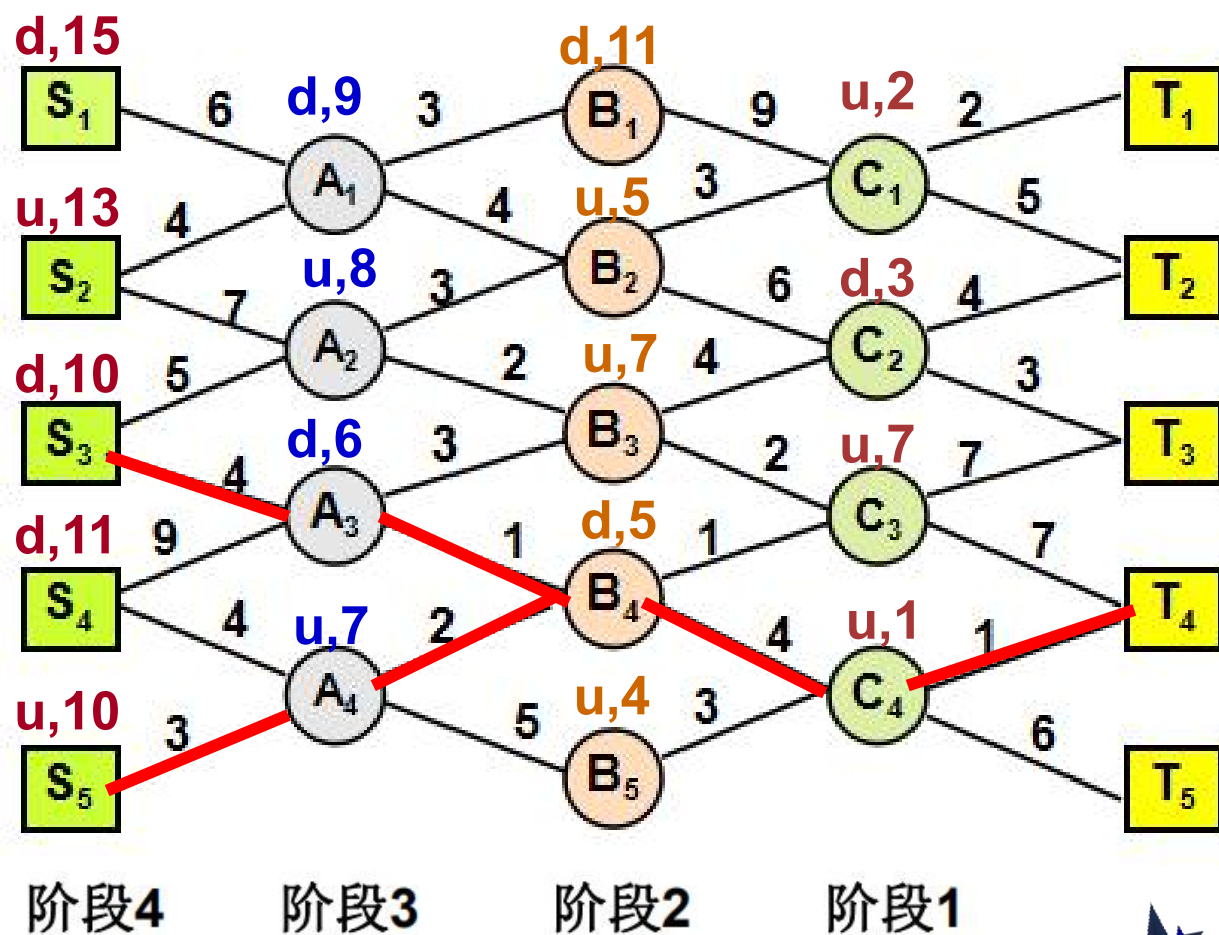
- 考察每一条从某个起点到某个终点的路径，计算长度，从其中找出最短路径。
- 在上述实例中，如果网络的层数为 k ，那么路径条数将接近于 2^k 。

(2) 动态规划算法：多阶段决策过程.

- 每步求解的问题是后面阶段求解问题的子问题。
- 每步决策将依赖于以前步骤的决策结果。

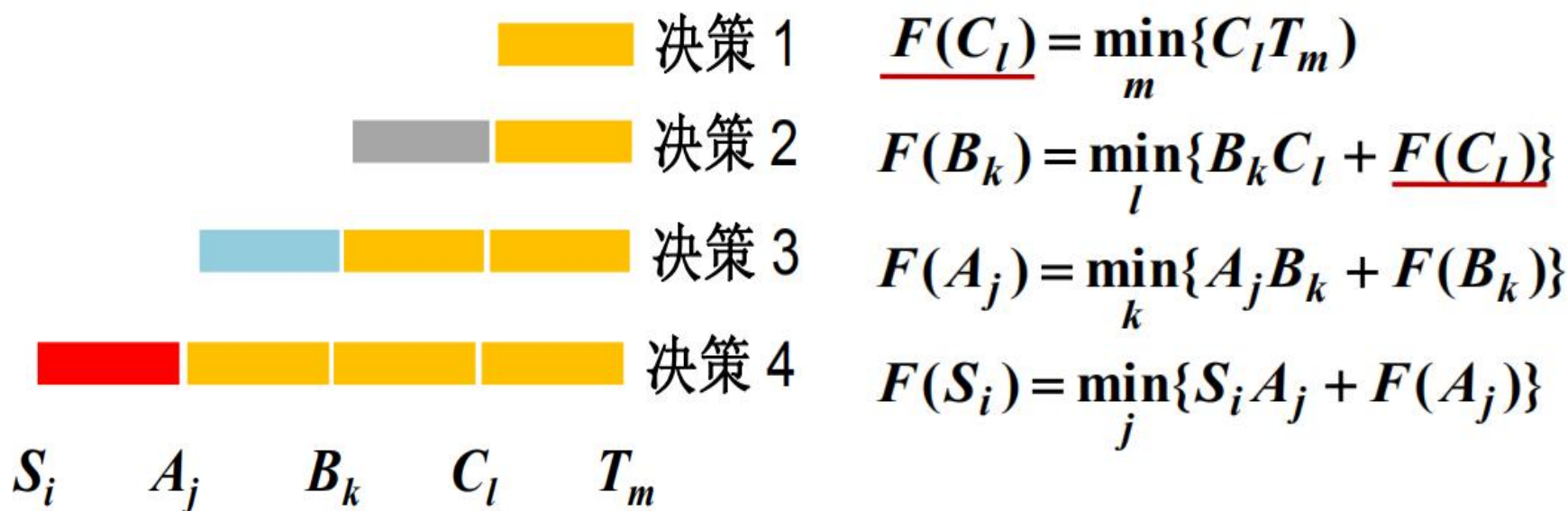


3.1 动态规划的设计思想

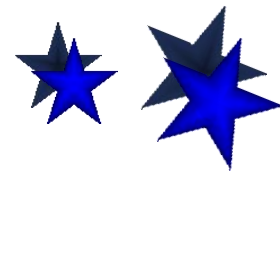


3.1 动态规划的设计思想

子问题界定



优化函数值之间存在依赖关系



3.1 动态规划的设计思想

□ 采用动态规划求解的问题的一般要具有3个性质

- **性质1—最优性原理：** 如果问题的最优解所包含的子问题的解也是最优的，就称该问题具有最优子结构，即满足最优性原理。



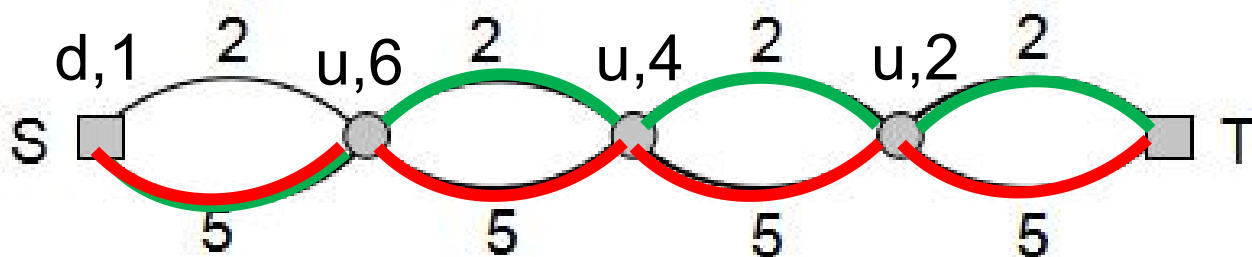
- **使用动态规划技术的条件：** 优化原则
- 优化原则：一个最优决策序列的任何子序列本身一定是相对于子序列的初始和结束状态的最优的决策序列



3.1 动态规划的设计思想

➤ 不符合最优子结构性质 (优化原则) 的例子

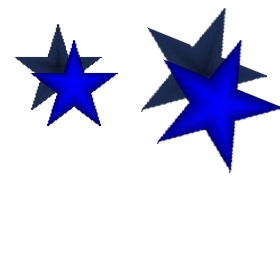
例：求总长模10的最小路径



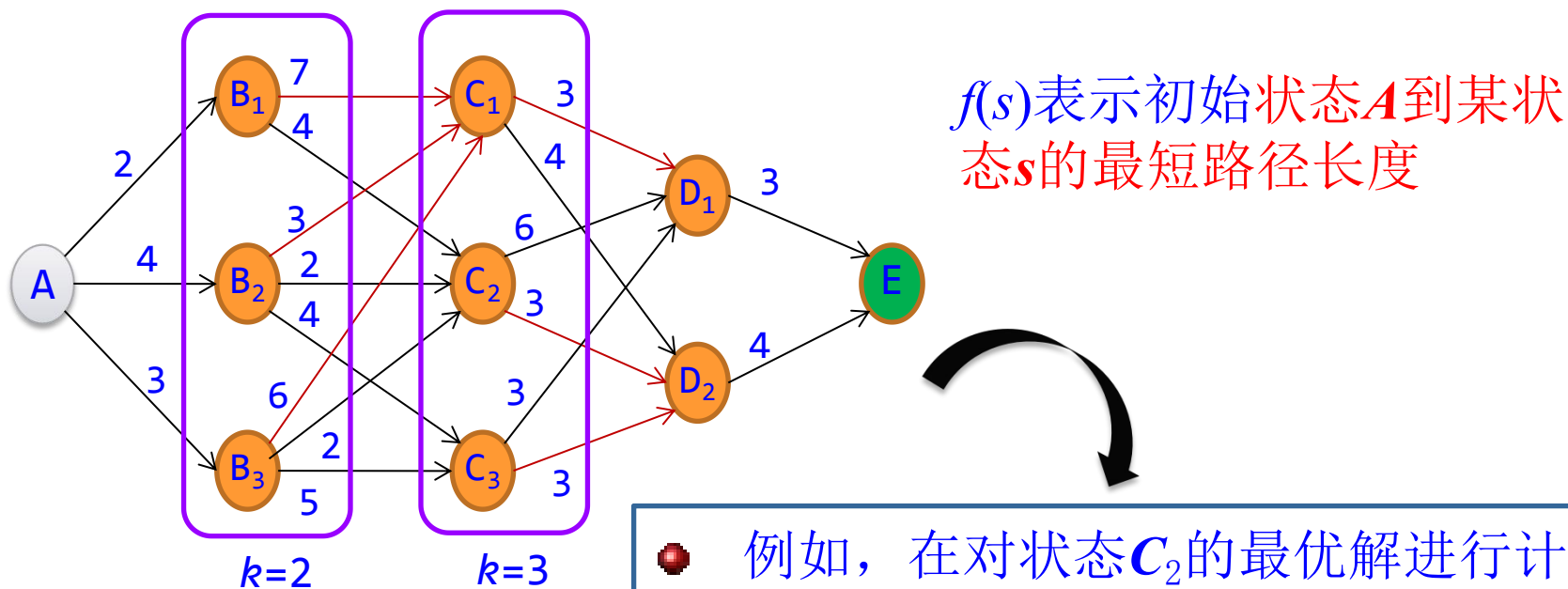
动态规划算法的解：下, 上, 上, 上

最优解：下, 下, 下, 下

不满足优化原则，不能用动态规划



- 性质2——无后效性：即某阶段中的某状态对应的最优解一旦确定，该状态以后的决策过程不会影响该状态，只与当前状态有关。



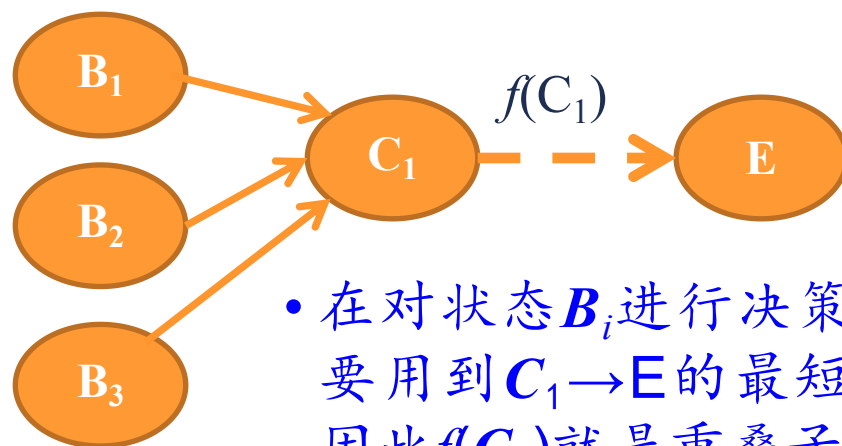
- 例如，在对状态 C_2 的最优解进行计算时，状态A, B_1 , B_2 , B_3 的最优解已经确定了。因此，在对 C_2 进行计算时，不会修改已经确定的最优解。

动态规划求解的基本步骤

- 性质3——有重叠子问题：即子问题之间是不独立的，一个子问题在下一阶段决策中可能被多次使用到。（该性质并不是动态规划的必要条件，但是如果没有这条性质，动态规划算法同其他算法相比就不具备优势）。

求斐波那契数列 $f(6)$ 时：

- 先求 $f(5)$ ，后求 $f(4)$
- 求 $f(5)$ 时，也要求 $f(4)$
- $f(4)$ 就是公共子问题
- 可以在某阶段求出 $f(4)$ 后保存到 dp 表中，以后再次使用时，查表即可。



- 在对状态 B_i 进行决策时，都要用到 $C_1 \rightarrow E$ 的最短路径，因此 $f(C_1)$ 就是重叠子问题。
- 子问题的解存于数组 dp 中，再次使用时查表即可。

动态规划求解的基本步骤

● 动态规划方案设计有着一定的模式，一般经历以下几个步骤

(1) 划分阶段：按照问题的时间或空间特征将问题划分成若干阶段。划分的阶段一定是有序的或可排序的，否则问题无法求解。

(2) 确定状态和状态变量：将问题发展到不同阶段所对应的情况，用不同的状态变量表示，同一阶段的不同状态看成一个状态集合。

(3) 确定决策并写出状态转移方程：根据问题所要求解的目标（决策）给出从上一个阶段到本阶段的状态转移方程。

(4) 寻找边界条件：因为状态转移方程是一个递推式，因此需要一个终止条件（边界条件）。

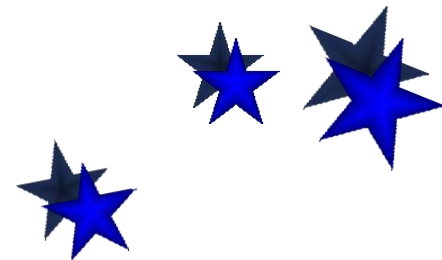
3.1 动态规划的设计思想

● 小结

(1) 动态规划(Dynamic Programming): 求解过程是多阶段决策过程, 每步处理一个子问题, 可用于求解组合优化问题。

(2) 求解步骤: 确定子问题的边界、从最小的子问题开始进行多步判断。

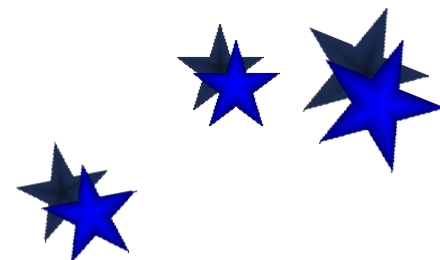
(3) 适用条件: 问题要满足优化原则或最优子结构性质, 即: 一个最优决策序列的任何子序列本身一定是相对于子序列的初始和结束状态的最优决策序列。



3.2 动态规划算法设计

■ 动态规划算法的设计要素

1. 问题建模，优化的目标函数是什么？约束条件是什么？
2. 如何划分子问题（边界）？
3. 问题的优化函数值与子问题的优化函数值存在着什么依赖关系？(递推方程)
4. 是否满足优化原则？
5. 最小子问题怎样界定？其优化函数值，即初值等于什么？



3.2 动态规划算法设计

■ 举例：矩阵链相乘

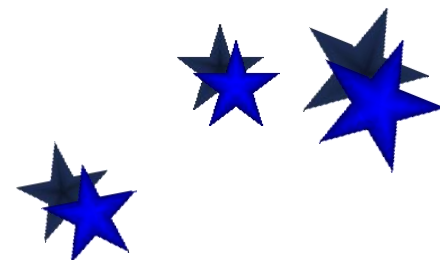
问题： 设 $A_1, A_2, \dots, A_n$ 为矩阵序列， A_i 为 $P_{i-1} \times P_i$ 阶矩阵， $i = 1, 2, \dots, n$. 试确定矩阵的乘法顺序，使得元素相乘的总次数最少.

输入： 向量

$$P = \langle P_0, P_1, \dots, P_n \rangle,$$

其中 $P_0, P_1, \dots, P_n$ 为 n 个矩阵的行数与列数

输出： 矩阵链乘法加括号的位置.



3.2 动态规划算法设计

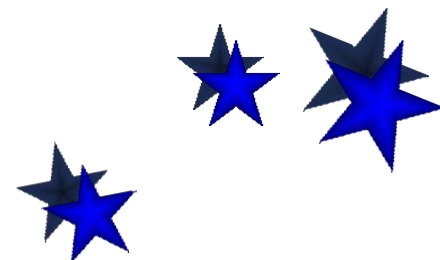
➤ 矩阵相乘基本运算次数

矩阵 A : i 行 j 列, B : j 行 k 列, 以元素相乘作基本运算, 计算 AB 的工作量

$$\begin{bmatrix} \dots \\ a_{t1} & a_{t2} & \dots & a_{tj} \\ \dots \end{bmatrix} \begin{bmatrix} b_{1s} \\ b_{2s} \\ \vdots \\ b_{js} \end{bmatrix} = \begin{bmatrix} c_{ts} \end{bmatrix}$$

$$c_{ts} = a_{t1} b_{1s} + a_{t2} b_{2s} + \dots + a_{tj} b_{js}$$

AB : i 行 k 列, 计算每个元素需要做 j 次乘法, 总计乘法次数为 ijk



3.2 动态规划算法设计

实例

实例: $P = \langle 10, 100, 5, 50 \rangle$

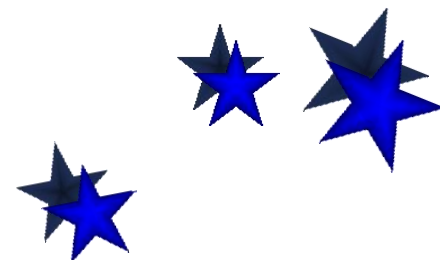
$A_1: 10 \times 100, A_2: 100 \times 5, A_3: 5 \times 50,$

乘法次序

$$\underline{(A_1 A_2)} A_3: 10 \times 100 \times 5 + 10 \times 5 \times 50 = \boxed{7500}$$

$$A_1 (A_2 A_3): 10 \times 100 \times 50 + 100 \times 5 \times 50 = 75000$$

第一种次序计算次数（乘法）最少



3.2 动态规划算法设计

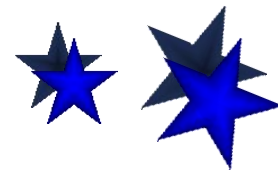
➤ 蛮力算法的时间复杂度分析

加 n 个括号的方法有 $\frac{1}{n+1} \binom{2n}{n}$ 种，
是一个Catalan数，是指数级别

$$W(n) = \Omega\left(\frac{1}{n+1} \frac{(2n)!}{n!n!}\right)$$

代入
Stirling
公式

$$= \Omega\left(\frac{1}{n+1} \frac{\sqrt{2\pi 2n} \left(\frac{2n}{e}\right)^{2n}}{\sqrt{2\pi n} \left(\frac{n}{e}\right)^n \sqrt{2\pi n} \left(\frac{n}{e}\right)^n}\right) = \Omega(2^{2n} / n^{\frac{3}{2}})$$



3.2 动态规划算法设计

➤ 动态规划算法实现

- 子问题划分

$A_{i..j}$: 矩阵链 $A_i A_{i+1} \dots A_j$, 边界 i, j

输入向量: $\langle P_{i-1}, P_i, \dots, P_j \rangle$

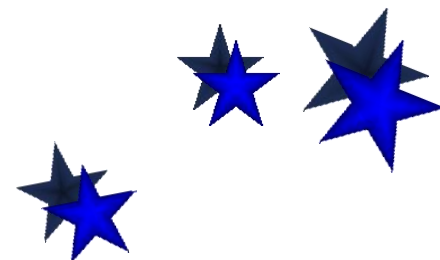
其最好划分的运算次数: $m[i, j]$

- 子问题的依赖关系

最优划分最后一次相乘发生在矩阵 k 的位置, 即

$$A_{i..j} = A_{i..k} A_{k+1..j}$$

$A_{i..j}$ 最优运算次数依赖于 $A_{i..k}$ 与 $A_{k+1..j}$ 的最优运算次数



3.2 动态规划算法设计

➤ 优化函数的递推方程

递推方程:

$m[i, j]$: 得到 $A_{i:j}$ 的最少的相乘次数

$m[i, j]$

$$= \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{ \underline{m[i, k]} + \underline{m[k+1, j]} + P_{i-1}P_kP_j \} & i < j \end{cases}$$

A_i	$\dots$	A_k	A_{k+1}	$\dots$	A_j
$P_{i-1} \times P_k$			$P_k \times P_j$		

该问题满足优化原则

设立标记函数: 为了确定加括号的次序, 设计表 $s[i, j]$, 记录求得最优时最后一次运算的划分位置。



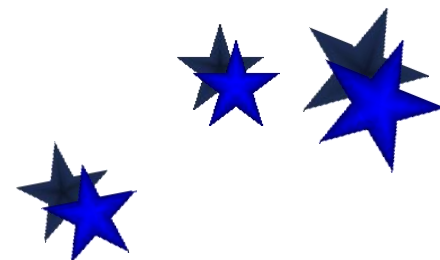
3.2 动态规划算法设计

- 动态规划算法设计要素小结

- (1) 多阶段决策过程，每步处理一个子问题，界定子问题的边界。

- (2) 列出优化函数的递推方程及初值

- (3) 问题要满足优化原则或最优子结构性质，即：一个最优决策序列的任何子序列本身一定是相对于子序列的初始和结束状态的最优决策序列。



3.2 动态规划算法设计

■ 动态规划算法的递归实现

➤ 矩阵相乘的递归算法

算法1 RecurMatrixChain (P, i, j)

子问
题 $i-j$

1. $m[i,j] \leftarrow \infty$

2. $s[i,j] \leftarrow i$

3. for $k \leftarrow i$ to $j-1$ do

划分
位置 k

4. $q \leftarrow$ RecurMatrixChain(P, i, k)
+ RecurMatrixChain($P, k+1, j$) + $p_{i-1}p_kp_j$

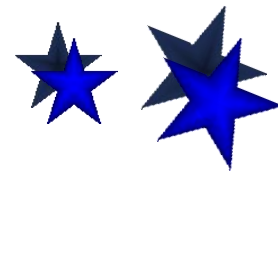
5. if $q < m[i,j]$

6. then $m[i,j] \leftarrow q$

7. $s[i,j] \leftarrow k$

8. return $m[i,j]$

找到更
好的解



3.2 动态规划算法设计

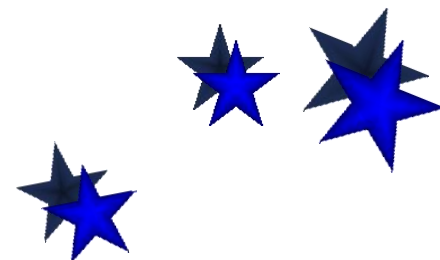
➤ 算法分析

- 时间复杂度的递推方程

$$T(n) \geq \begin{cases} O(1) & n = 1 \\ \sum_{k=1}^{n-1} (T(k) + T(n-k) + O(1)) & n > 1 \end{cases}$$

$$T(n) \geq O(n) + \sum_{k=1}^{n-1} T(k) + \sum_{k=1}^{n-1} T(n-k)$$

$$T(n) \geq O(n) + 2 \sum_{k=1}^{n-1} T(k)$$



3.2 动态规划算法设计

- 时间复杂度: $T(n) \geq 2^{n-1}$

数学归纳法证明:

$n=2$, 显然为真.

假设对于任何小于 n 的 k , 命题为真

证明 $k=n$ 时命题成立:

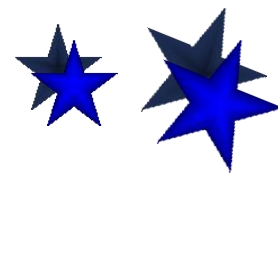
$$T(n) \geq O(n) + 2 \sum_{k=1}^{n-1} T(k)$$

代入归
纳假设

$$\geq O(n) + 2 \sum_{k=1}^{n-1} 2^{k-1}$$

等比数
列求和

$$= O(n) + 2(2^{n-1} - 1) \geq 2^{n-1}$$



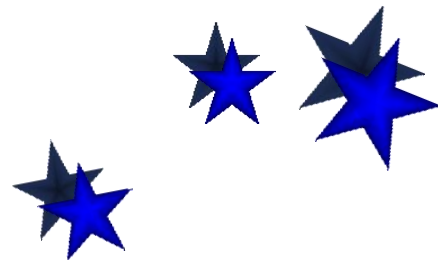
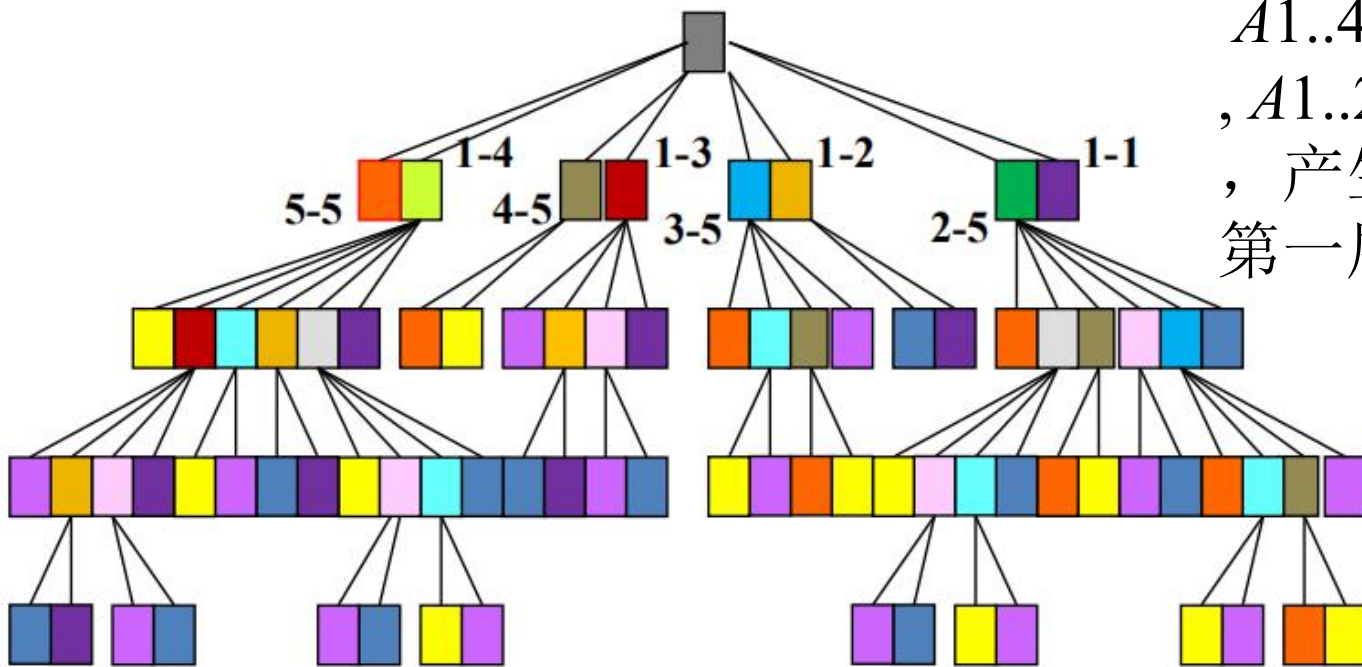
3.2 动态规划算法设计

➤ 递归实现时间复杂度高的原因分析

子问题的产生, $n=5$

划分:

$A_{1..4} A_{5..5}$, $A_{1..3} A_{4..5}$,
 $A_{1..2} A_{3..5}$, $A_{1..1} A_{2..5}$,
产生 8 个子问题, 即
第一层的 8 个结点.



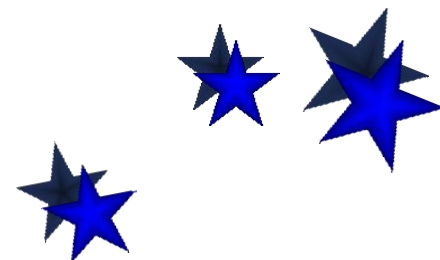
3.2 动态规划算法设计

子问题	1-1	2-2	3-3	4-4	5-5	1-2	2-3	3-4	4-5	1-3	2-4	3-5	1-4	2-5	1-5
数	8	12	14	12	8	4	5	5	4	2	2	2	1	1	1

$n=5$, 计算子问题: 81个; 不同的子问题: 15个

复杂度高的原因: 相同的子问题需要重复计算

解决方案: 使用恰当的数据结构存储子问题的结果, 以避免重复计算。



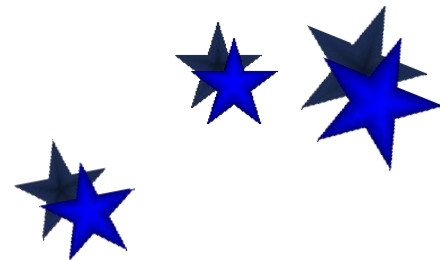
3.2 动态规划算法设计

- 动态规划递归实现小结

(1) 与蛮力算法相比较，动态规划算法利用了子问题优化函数间的依赖关系，时间复杂度有所降低。

(2) 动态规划算法的递归实现效率不高，原因在于同一子问题多次重复出现，每次出现都需要重新计算一遍。

(3) 采用空间换时间策略，记录每个子问题首次计算结果，后面再用时就直接取值，每个子问题只算一次。

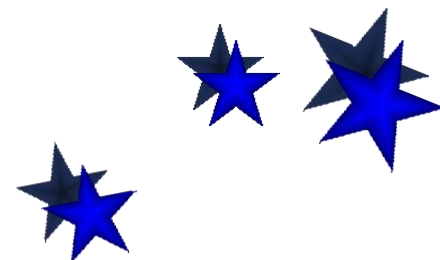


3.2 动态规划算法设计

■ 动态规划算法的迭代实现

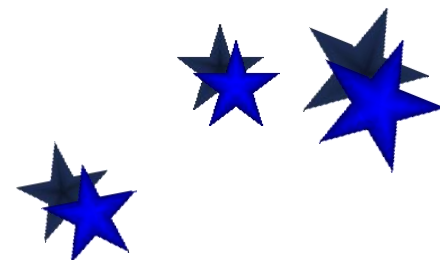
➤ 迭代计算的关键

- 每个子问题只计算一次
- 迭代过程
 - 从最小的子问题算起
 - 考虑计算顺序，以保证后面用到的值前面已经计算好
 - 存储结构保存计算结果——备忘录
- 解的追踪
 - 设计标记函数标记每步的决策
 - 考虑根据标记函数追踪解的算法



3.2 动态规划算法设计

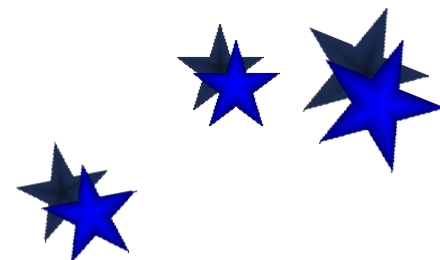
- 矩阵链乘法不同规模子问题个数分析
 - 长度1: 只含1个矩阵, 有 n 个子问题(不需要计算)
 - 长度2: 含2个矩阵, $n-1$ 个子问题
 - 长度3: 含3个矩阵, $n-2$ 个子问题
 - ...
 - 长度 $n-1$: 含 $n-1$ 个矩阵, 2个子问题
 - 长度 n : 原始问题, 只有1个



3.2 动态规划算法设计

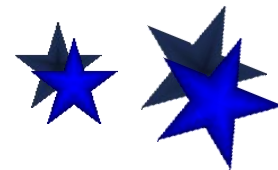
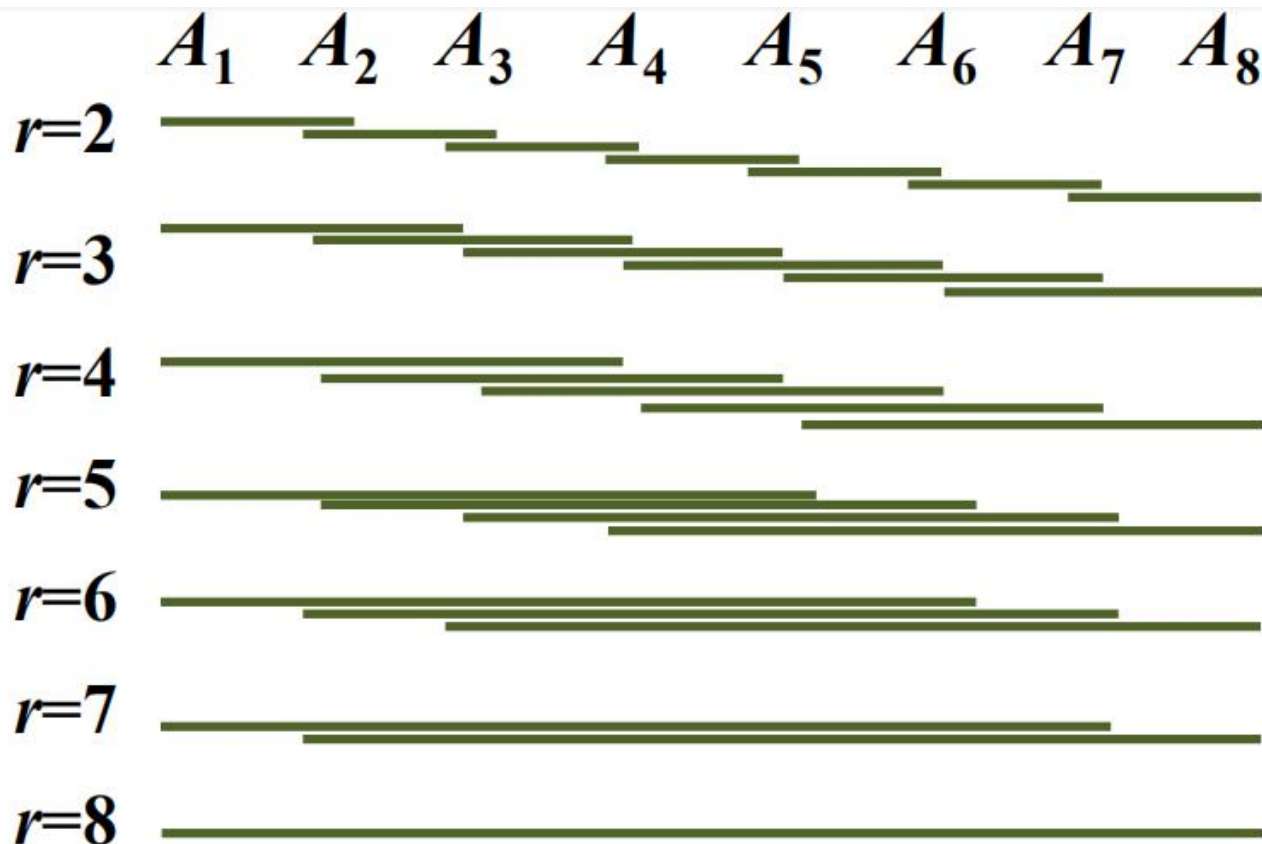
➤ 矩阵链乘法迭代顺序

- 长度为1: 初值, $m[i, i] = 0$
- 长度为2: $1..2, 2..3, 3..4, \dots, n-1..n$
- 长度为3: $1..3, 2..4, 3..5, \dots, n-2..n$
- ...
- 长度为 $n-1$: $1..n-1, 2..n$
- 长度为 n : $1..n$



3.2 动态规划算法设计

➤ $n=8$ 的子问题计算顺序



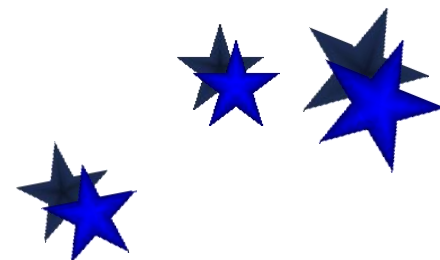
3.2 动态规划算法设计

➤ 实例

输入： $P = \langle 30, 35, 15, 5, 10, 20 \rangle$,
 $n = 5$

矩阵链： $A_1 A_2 A_3 A_4 A_5$ ，其中
 $A_1: 30 \times 35$, $A_2: 35 \times 15$, $A_3: 15 \times 5$,
 $A_4: 5 \times 10$, $A_5: 10 \times 20$

备忘录： 存储所有子问题的最小乘法次数及得到这个值的划分位置。



3.2 动态规划算法设计

备忘录：矩阵 m 标记函数： s

$r=1$	$m[1,1] = 0$	$m[2,2] = 0$	$m[3,3] = 0$	$m[4,4] = 0$	$m[5,5] = 0$
$r=2$	$m[1,2] = 15750$	$m[2,3] = 2625$	$m[3,4] = 750$	$m[4,5] = 1000$	
$r=3$	$m[1,3] = 7875$	$m[2,4] = 4375$	$m[3,5] = 2500$		
$r=4$	$m[1,4] = 9375$	$m[2,5] = 7125$			
$r=5$	$m[1,5] = 11875$				

$r=2$	$s[1,2] = 1$	$s[2,3] = 2$	$s[3,4] = 3$	$s[4,5] = 4$	
$r=3$	$s[1,3] = 1$	$s[2,4] = 3$	$s[3,5] = 3$		
$r=4$	$s[1,4] = 3$	$s[2,5] = 3$			
$r=5$	$s[1,5] = 3$				



3.2 动态规划算法设计

➤ 例如： $m[2,5]$ 代表矩阵： $A_2A_3A_4A_5$ ，括号的添加有如下可能：

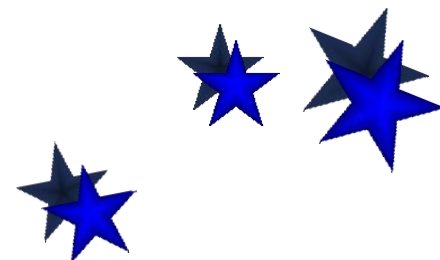
$$(1) (A_2)A_3A_4A_5=0+m[3,5]+35 \times 15 \times 20=13000$$

$$(2) (A_2A_3)A_4A_5=m[2,3]+m[4,5]+35 \times 5 \times 20=7125$$

$$(3) (A_2A_3A_4)A_5=m[2,4]+0+35 \times 10 \times 20=11375$$

$$(4) m[2,5]=\min\{13000,7125,11375\}=7125$$

$$\times s[2,5]=3$$



3.2 动态规划算法设计

➤ 解的追踪:

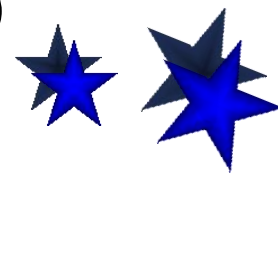
(1) 最少的乘法次数: $m[1,5]=11875$

(2) 加括号的位置

解的追踪: $s[1,5]=3 \Rightarrow (A_1A_2A_3)(A_4A_5)$

$s[1,3]=1 \Rightarrow A_1(A_2A_3)$

$A_1A_2A_3A_4A_5$ 的计算顺序为: $(A_1(A_2A_3))(A_4A_5)$

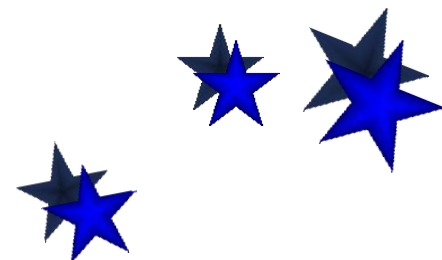


3.2 动态规划算法设计

➤ 迭代实现: **MatrixChain(P, n)**

1. 令所有的 $m[i, i]$ 初值为0
2. for $r \leftarrow 2$ to n do // r 为链长
3. for $i \leftarrow 1$ to $n - r + 1$ do // 左边界 i
4. $j \leftarrow i + r - 1$ // 右边界 j
5. $m[i, j] \leftarrow m[i + 1, j] + p_{i-1} p_i p_j$ // $k = i$
6. $s[i, j] \leftarrow i$ // 记录 k
7. for $k \leftarrow i + 1$ to $j - 1$ do // 遍历 k
8. $t \leftarrow \underline{m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j}$
9. if $t < m[i, j]$
10. then $m[i, j] \leftarrow t$ // 更新解
11. $s[i, j] \leftarrow k$

二维数组 m
与 s 为备忘录



3.2 动态规划算法设计

➤ 时间复杂度分析

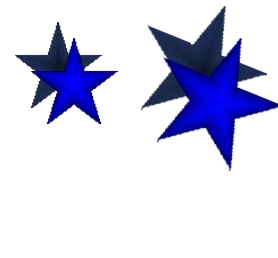
- **根据伪码**：行 2, 3, 7 都是 $O(n)$ ，循环执行 $O(n^3)$ 次，内部为 $O(1)$

$$W(n) = O(n^3)$$

- **根据备忘录**：估计每项工作量，求和。子问题有 $O(n^2)$ 个，确定每个子问题的最少乘法次数需要对不同划分位置比较，需要 $O(n)$ 时间。

$$W(n) = O(n^3)$$

- 追踪解工作量 $O(n)$ ，总工作量 $O(n^3)$ 。



3.2 动态规划算法设计

➤ 两种实现的比较

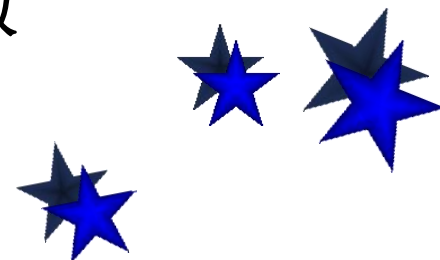
递归实现：时间复杂性高，空间较小

迭代实现：时间复杂性低，空间消耗多

原因：递归实现子问题多次重复计算，子问题计算次数呈指数增长. 迭代实现每个子问题只计算一次.

动态规划时间复杂度：

备忘录各项计算量之和 + 追踪解工作量. 通常追踪工作量不超过计算工作量，是问题规模的多项式函数



3.2 动态规划算法设计

➤ 动态规划算法的要素:

- 划分子问题, 确定子问题边界, 将问题求解转 变成多步判断的过程.
- 定义优化函数, 以该函数极大(或极小) 值作为依据, 确定是否满足优化原则.
- 列优化函数的递推方程和边界条件
- 自底向上计算, 设计备忘录 (表格) • 考虑是否需要设立标记函数
- 用递推方程或备忘录估计时间复杂度

